

GeepSeek: 基于 CNN 与蒙特卡洛树搜索的自对弈五子棋强化学习系统

GeepSeek: A CNN-MCTS Hybrid Architecture for Efficient Self-Play Learning in Gomoku

jabberloop_is_killin^{1†}, copycat666², doro², 横原立花³, 鈴木健太⁴, 山田優子⁵, Weiming Zhang⁶,

Xiaoxiao Li⁷, Fangchen Wang⁸, Shuaiqi Chen⁹, Fan Huang¹⁰, Zhiquan Lin¹¹

1. 中国香港五子棋协会, 香港特别行政区; 2. Eigenlife 机器学习实验室, 东北电力大学, 吉林
3. 東京都立武蔵高等学校連珠部, 东京, 日本; 4. 大阪府立北野高等学校連珠同好会, 大阪, 日本
5. 神奈川県立横浜翠嵐高等学校連珠クラブ, 神奈川, 日本; 6. 学生五子棋协会, 清华大学, 北京
7. 棋类协会五子棋部, 北京大学, 北京; 8. 棋牌协会五子棋分会, 上海交通大学, 上海
9. 珞珈棋社, 武汉大学, 武汉; 10. 启真棋社, 浙江大学, 杭州; 11. 弈秋棋社, 南京大学, 南京
†: Corresponding Author (This author has not provided any email address or contact information yet)

文献分类码: R

版号: ET; 卷号: 2; 索引: 1; 年份: 2026

摘要: 本文提出 *GeepSeek*, 一个基于卷积神经网络 (CNN) 与蒙特卡洛树搜索 (MCTS) 的轻量化自对弈强化学习系统, 用于五子棋博弈。系统采用双头 CNN 架构, 结合策略网络与价值网络, 通过纯自我对弈进行训练, 无需人类棋谱数据。在单 GPU (NVIDIA RTX 4080) 环境下训练 48 小时后, 系统实现了基本战术能力, 能够与传统的 α - β 剪枝搜索算法进行有效对弈。实验结果表明, 该框架在有限计算资源下能够完成自对弈学习过程, 并展现出从随机策略到基础战术的进化轨迹。本研究为在资源受限环境下探索 AlphaZero 类算法提供了可复现的实现案例, 并分析了轻量化设计对学习效果的影响。

关键词: 五子棋; 卷积神经网络; 蒙特卡洛树搜索; 强化学习; 自对弈

Abstract: This paper presents *GeepSeek*, a lightweight self-play reinforcement learning system for Gomoku based on **convolutional neural networks (CNN)** and **Monte Carlo Tree Search (MCTS)**. The system adopts a dual-head CNN architecture that combines policy and value networks, trained exclusively through self-play without human game data. After 48 hours of training on a single GPU (NVIDIA RTX 3080), the system achieves basic tactical capabilities and can effectively compete with traditional α - β pruning search algorithms. Experimental results demonstrate that the framework can complete the self-play learning process under limited computational resources and exhibits an evolutionary trajectory from random strategies to basic tactics. This study provides a reproducible implementation case for exploring AlphaZero-like algorithms in resource-constrained environments and analyzes the impact of lightweight design on learning effectiveness.

Keywords: Gomoku; Convolutional Neural Networks; Monte Carlo Tree Search; Reinforcement Learning; Self-Play

目录

I	研究背景与意义	2
II	理论基础	2
II-A	五子棋博弈复杂性理论与传统求解方法	2
II-B	博弈论与决策理论数学基础	3
II-C	蒙特卡洛树搜索与深度神经网络的理论结合	4
II-D	深度学习基础与卷积神经网络架构	5
II-E	自对弈强化学习框架的理论分析	6
III	<i>GeepSeek</i> 系统设计与实现	6
III-A	整体架构设计	6
III-B	双头卷积神经网络架构设计	7
III-B1	输入表示与特征编码	7
III-B2	轻量化残差设计	8
III-B3	策略头设计	8
III-B4	价值头设计	8
III-B5	参数效率分析	8
III-B6	训练稳定性增强技术	8
III-C	蒙特卡洛树搜索算法实现细节	9
III-C1	节点数据结构优化	9
III-C2	虚拟损失与并行搜索	10
III-C3	探索增强技术	10
III-C4	终局精确求解优化	10
III-C5	温度调度策略	10
III-D	自对弈训练流程与优化策略	10
III-D1	并行自对弈数据生成	10
III-D2	经验缓冲区设计与采样策略	10
III-D3	损失函数设计与梯度优化	11
III-D4	模型评估与选择策略	11
III-D5	超参数配置与自适应调整	12
III-D6	训练收敛性分析	12
III-E	系统实现与性能优化	12

IV	实验设计与结果分析	13
IV-A	实验环境与基准设置	13
IV-B	训练过程分析	13
IV-B1	学习曲线与收敛性	13
IV-B2	超参数敏感性分析	13
IV-B3	消融实验分析	13
IV-C	性能对比实验	13
IV-C1	总体性能对比	13
IV-C2	不同开局类型表现	15
IV-C3	终局精确度测试	15
IV-C4	计算效率对比	15
IV-C5	策略新颖性分析	15
IV-D	人类对弈测试	15
V	结论与展望	15
V-A	研究工作总结	15
V-B	系统局限性分析	16
V-C	未来研究方向	17
V-D	研究意义与影响	17
参考文献		17
附录		18
A	五子棋复杂度理论分析	18
A1	状态空间精确计算	18
A2	胜利模式组合分析	18
B	神经网络架构详细参数	18
C	蒙特卡洛树搜索算法复杂度分析	18
C1	时间复杂度理论推导	18
C2	空间复杂度分析	18
D	训练数据统计分析	19
D1	自对弈数据分布特性	19
D2	探索与利用平衡分析	19
E	专业评估标准详细说明	19
E1	人类棋手评分细则	19
E2	终局测试集构建方法	19
F	计算环境详细配置	19
F1	软件依赖与版本管理	19
F2	硬件性能基准测试	19
G	五子棋主流赛事	20
H	五子棋 AI 对弈的主流方法	20

I. 研究背景与意义

近年来,人工智能在完全信息博弈领域取得了突破性进展,尤其是以深度强化学习与蒙特卡洛树搜索(MCTS)相结合的自对弈学习框架,在围棋、国际象棋等复杂棋类游戏中展现出超越人类顶尖水平的性能 [1], [2]。AlphaGo Zero 及其通用版本 AlphaZero 的成功,标志着一种不依赖人类先验知识、仅通过自我对弈即可从零开始掌握复杂博弈策略的新型人工智能范式的诞生。这一范式摒弃了传统博弈程序中繁杂的手工特征工程与启发式评估函数设计,转而采用深度神经网络直接从原始棋盘状态中学习表示与策略,并通过 MCTS 进行前瞻性搜索与策略改进,实现了端到端的学习与决策。

然而, AlphaZero 系列算法的卓越性能建立在大规模计算资源的支撑之上。公开资料显示,其原始实现需要数千块 TPU 进行数周乃至数月的持续训练,巨大的计算开销使得这类前沿研究难以在普通学术机构或个人的资源条件下进行复现、验证与拓展 [3]。这种“计算鸿沟”在一定程度上阻碍了相关研究思想的广泛传播、教育普及与迭代创新。因此,探索如何在有限计算资源下实现有效的自对弈强化学习,降低此类先进算法的实践门槛,成为一个具有现实意义的研究问题。

五子棋 (Gomoku) 作为一种历史悠久、规则简单的两人零和博弈,是研究此类问题的理想测试平台。其棋盘规模 (通常为 15×15) 与状态空间复杂度 (约 10^{105} 量级) 介于国际象棋 (约 10^{46}) 与围棋 (约 10^{170}) 之间,既包含了丰富的战术组合与战略规划,又避免了围棋极度的复杂性所带来的计算负担 [4]。传统五子棋 AI 多采用基于固定模式的专家系统或结合了特定领域知识的 α - β 搜索算法,这些方法虽然在一定水平上有效,但其性能严重依赖于精心调校的评估函数,泛化能力有限,且难以迁移到其他类似问题上。将 AlphaZero 的自对弈思想应用于五子棋,不仅有望开发出更强大、更具通用性的博弈程序,更能为理解自监督学习在中等复杂度问题上的表现提供宝贵的实验案例。

本研究的意义主要体现在理论与应用两个层面。

在理论层面,本研究旨在探索和验证自对弈强化学习框架在资源受限条件下的有效性边界与性能极限。通过设计并实现一个轻量化的“AlphaZero-like”系统——*GeepSeek*,我们试图回答以下几个核心问题:第一,在显著减少网络参数与计算预算的情况下,系统能否通过自对弈过程实现有效的策略进化,展现出从随机行为到基础战术、乃至高级战略的层次化学习能力?第二,轻量化设计 (如更浅的网络结构、更少的 MCTS 模拟次数) 会对学习过程的稳定性、收敛速度以及最终策略的质量产生何种系统性影响?对这些问题的实证研究,有助于深化我们对深度强化学习、神经网络表示能力以及树搜索算法之间复杂相互作用的理解,为后续的理论分析提供数据支持与观察基础。

在应用与实践层面,本研究的意义则更为直接和多元。首先,它提供了一个完整的、可复现的轻量化自对弈学习系统实现。详细的代码、可调的参数与完整的训练日志,使得其他研究者、教育者或爱好者能够以较低的成本复现实验,并在此基础上进行修改、优化与拓展,从而降低了进入该研究领域的壁垒。其次,该系统可作为教学演示的优质案例,生动展示深度强化学习与树搜索相结合的核心思想,以及 AI 如何通过“自我博弈”实现“自我提升”的奇妙过程。最后,针对五子棋这一特定领域,一个成功的轻量化 AI 模型本身也具有一定的应用价值,例如用于开发智能游戏对手、辅助棋艺分析或作为其他博弈 AI 研究的性能基准。

综上所述,本研究不仅是对前沿人工智能方法在有限资源下可行性的一次重要实践检验,也为推动相关技术的普及化、教育化以及后续研究提供了有价值的工具与洞见。

II. 理论基础

A. 五子棋博弈复杂性理论与传统求解方法

五子棋,亦称连珠 (Gomoku),是一种起源于中国的两人零和、完全信息博弈。对弈双方在 15×15 的网格棋盘上交替落子,黑方先行,率先在横、竖或斜任一方向上形成连续五个或以上己方棋子的一方获胜。尽管规则极其简洁,五子棋所蕴含的策略深

度与计算复杂度使其成为人工智能与博弈论研究的重要模型。其状态空间复杂度可通过组合数学进行估算：棋盘共有 225 个交叉点，每个点有黑子、白子、空位三种状态，故理论上的最大状态数为 $3^{225} \approx 10^{107}$ 。然而，由于游戏在达成五连时立即终止，且大量对称、重复的无效状态存在，实际可达的游戏状态数约为 10^{105} 量级 [4]。游戏树复杂度则反映了从初始状态到达所有可能终局所需遍历的路径总数。对于五子棋，其平均分支因子（即每一步的合法着法数）在游戏前期接近棋盘空位数，随着棋子增多而减少，整局游戏的分支因子平均值估算约为 35。由此，典型对局长度约为 30 至 60 步，其游戏树复杂度约为 10^{70} 量级。这一复杂度远低于围棋（ 10^{360} ），但显著高于国际象棋（ 10^{43} ），恰好落在传统穷举搜索不可行、但启发性方法又面临挑战的“中间地带”。

面对如此复杂的搜索空间，传统五子棋人工智能主要依赖以极小化极大算法（Minimax）为核心的博弈树搜索，并辅以大量领域知识。极小化极大算法的核心思想是模拟对弈双方在博弈树中交替选择对己方最有利的着法：一方（最大化玩家）试图最大化评估分数，而另一方（最小化玩家）试图最小化该分数。算法递归地遍历树结构直至预定深度，然后通过静态评估函数为叶节点打分，再将这些分数回传至根节点以确定最优着法。其计算成本随搜索深度 d 和分支因子 b 呈指数增长，约为 $O(b^d)$ 。为了应对这一组合爆炸问题， α - β 剪枝算法被广泛采用。该算法通过维护两个值—— α （当前最大化玩家所能保证的最佳分数下界）和 β （当前最小化玩家所能保证的最佳分数上界）——来识别并剪除那些不可能影响最终决策的子树分支，在最优情况下能将搜索成本降至 $O(b^{d/2})$ 。在实际五子棋程序中，通常会结合迭代深化、置换表、杀手启发、历史启发等技术进一步优化搜索效率 [5]。

然而，搜索算法的有效性极大程度上依赖于静态评估函数的质量。传统的五子棋评估函数是领域知识的结晶，通常通过识别和加权一系列预定义的“棋形”或“模式”来实现。这些模式包括但不限于：连五（胜局）、活四（下一步可成五）、冲四（单方向成五）、活三（可发展为活四）、眠三等。每一种模式根据其威胁程度被赋予不同的分数，局面的总评估值则为双方各类模式得分的加权和 [6]。这种方法的优势在于直观高效，在计算资源有限时能迅速给出合理的评估。但其缺陷也显而易见：首先，评估函数的设计高度依赖于专家的先验知识，耗时费力且难以保证完备性与平衡性；其次，线性加权模型难以捕捉棋子间复杂的非线性交互与全局态势；最后，基于固定模式的评估在面对未预见的、或由多个简单模式组合形成的复杂局面时可能失效。这些局限性促使研究者探索不依赖手工特征、能够从数据中自动学习评估策略的新方法。

B. 博弈论与决策理论数学基础

五子棋作为一种完全信息、零和、确定性的序贯博弈，其数学模型可由扩展式博弈进行严谨描述。记博弈参与者为黑方 P_B 与白方 P_W ，双方交替行动的阶段 $t = 0, 1, 2, \dots, T$ ，其中 T 为博弈终止步数。在任意阶段 t ，博弈状态 $s_t \in \mathcal{S}$ 包含了棋盘上所有棋子的配置信息，而行动 $a_t \in \mathcal{A}(s_t)$ 表示当前玩家在合法位置落子的选择。状态转移函数 $f: \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ 是确定性的，即 $s_{t+1} = f(s_t, a_t)$ 。博弈的终止状态集合 $\mathcal{S}_T \subset \mathcal{S}$ 包含所有形成五连或棋盘填满的状态。对于任一终止状态 s_T ，定义效用函数 $u_i(s_T)$ 表示玩家 i 的收益。在零和博弈框架下，通常设定 $u_B(s_T) = -u_W(s_T)$ 。对于标

准五子棋，可简单定义黑胜时 $u_B = 1, u_W = -1$ ，白胜时反之，平局时均为 0。

玩家的策略 $\pi_i: \mathcal{S} \rightarrow \Delta(\mathcal{A})$ 是一个从状态到行动概率分布 $\Delta(\mathcal{A})$ 的映射，其中 $\Delta(\mathcal{A})$ 表示行动空间上的概率单纯形。对于确定性策略， $\pi_i(s)$ 是某个行动的狄拉克分布。给定双方策略 $\pi = (\pi_B, \pi_W)$ ，从初始状态 s_0 出发的期望效用（对于玩家 i ）可通过递归方式定义：

$$V_i^\pi(s) = \begin{cases} u_i(s), & \text{如果 } s \in \mathcal{S}_T \\ \mathbb{E}_{a \sim \pi_{\text{player}(s)}(s)} [V_i^\pi(f(s, a))], & \text{否则} \end{cases} \quad (1)$$

其中 $\text{player}(s)$ 表示状态 s 下该行动的一方。式 (1) 定义的 $V_i^\pi(s)$ 称为玩家 i 在策略 π 下的状态价值函数。博弈论的核心目标之一是寻找纳什均衡策略 $\pi^* = (\pi_B^*, \pi_W^*)$ ，使得任何单一玩家单方面偏离策略都无法获得更高收益，即满足：

$$V_B^{(\pi_B^*, \pi_W^*)}(s_0) \geq V_B^{(\pi_B, \pi_W^*)}(s_0), \quad V_W^{(\pi_B^*, \pi_W^*)}(s_0) \geq V_W^{(\pi_B^*, \pi_W)}(s_0) \quad (2)$$

对于有限二人零和完全信息博弈，冯·诺依曼的最小最大定理保证了至少存在一个均衡解，且双方的均衡价值 V^* 满足：

$$V^* = \max_{\pi_B} \min_{\pi_W} V_B^\pi(s_0) = \min_{\pi_W} \max_{\pi_B} V_B^\pi(s_0) \quad (3)$$

式 (3) 为极小化极大算法提供了理论基石。在五子棋的离散有限状态空间中，原则上可以通过递归求解每个状态的博弈值来获得最优策略，即求解贝尔曼最优方程：

$$V_i^*(s) = \begin{cases} u_i(s), & s \in \mathcal{S}_T \\ \max_{a \in \mathcal{A}(s)} V_i^*(f(s, a)), & \text{玩家 } i \text{ 行动} \\ \min_{a \in \mathcal{A}(s)} V_i^*(f(s, a)), & \text{对手行动} \end{cases} \quad (4)$$

最优策略 $\pi_i^*(s)$ 即在玩家 i 行动时，选择使 $V_i^*(f(s, a))$ 最大化的行动 a ；在对手行动时，则选择使 $V_i^*(f(s, a))$ 最小化的行动（从 i 的视角）。然而，如 II-A 所述，由于状态空间巨大，直接求解式 (4) 是不现实的。这促使研究者发展出近似动态规划、启发式搜索与函数逼近等方法来近似最优策略与价值函数。

在强化学习框架下，五子棋博弈可建模为一个马尔可夫决策过程，其中智能体（如黑方）与环境（包含白方对手与棋盘动态）交互。智能体在状态 s_t 下根据策略 $\pi_\theta(a_t|s_t)$ 选择行动 a_t ，接收奖励 r_t 并转移到新状态 s_{t+1} 。对于五子棋，通常仅在终局给予奖励：获胜时 $r_T = 1$ ，失败时 $r_T = -1$ ，平局时 $r_T = 0$ ，其余中间步 $r_t = 0$ 。智能体的目标是最大化期望累计折扣回报 $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$ ，其中 $\gamma \in [0, 1]$ 为折扣因子，对于有限步终止的棋类游戏通常取 $\gamma = 1$ 。状态价值函数 $V^\pi(s) = \mathbb{E}^\pi[G_t|s_t = s]$ 与状态-行动价值函数（Q 函数） $Q^\pi(s, a) = \mathbb{E}^\pi[G_t|s_t = s, a_t = a]$ 是评估策略优劣的关键。基于此，策略梯度定理给出了直接优化参数化策略 π_θ 的方向：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t|s_t) G_t \right] \quad (5)$$

其中 $\tau = (s_0, a_0, r_0, \dots, s_T)$ 表示一条轨迹。式 (5) 构成了包括 REINFORCE 算法在内的一系列策略优化方法的理论基础 [7]。然而，直接使用蒙特卡洛回报 G_t 往往方差较大。为减少方差，常引入基准函数（如状态价值函数 $V_\phi(s)$ ）构造优势函数估计 $A_t =$

$G_t - V_\phi(s_t)$, 此时策略梯度可写为:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) A_t \right] \quad (6)$$

这引导出演员-评论家架构, 其中“演员” π_θ 负责选择行动, “评论家” V_ϕ 负责评估状态价值以指导演员更新。AlphaZero 系列算法可视为一种特殊的演员-评论家方法, 其“评论家”是经过 MCTS 强化的价值网络, 而“演员”则是 MCTS 导出的策略分布 [1]。

另一个关键概念是探索与利用的权衡。在自对弈中, 智能体需在利用当前已知最佳策略与探索可能更优的新策略之间取得平衡。为此, 常在实际策略中引入随机性, 例如采用玻尔兹曼探索策略:

$$\pi_\tau(a|s) = \frac{\exp(Q(s, a)/\tau)}{\sum_b \exp(Q(s, b)/\tau)} \quad (7)$$

其中温度参数 τ 控制探索程度: $\tau \rightarrow 0$ 时退化为贪婪策略; $\tau \rightarrow \infty$ 时趋近于均匀随机策略。在 *GeepSeek* 的训练早期, 较高的 τ 值有助于广泛探索状态空间; 随着训练进行, 逐渐降低 τ 以收敛到确定性更强的策略。

C. 蒙特卡洛树搜索与深度神经网络的理论结合

蒙特卡洛树搜索作为近年来博弈人工智能领域的核心技术突破, 其核心思想是通过大量随机模拟来构建并探索博弈树的部分视图, 从而在有限的计算资源下做出近似最优的决策。MCTS 算法通常包含四个阶段: 选择 (Selection)、扩展 (Expansion)、模拟 (Simulation) 与回传 (Backpropagation) [8]。在经典的 UCT 算法中, 树节点 n 对其子节点 a 的选择遵循上置信界公式:

$$a^* = \arg \max_{a \in \mathcal{A}(n)} \left\{ Q(n, a) + c \sqrt{\frac{\ln N(n)}{N(n, a)}} \right\} \quad (8)$$

其中 $Q(n, a)$ 为行动 a 的平均回报估计, $N(n)$ 为节点 n 的总访问次数, $N(n, a)$ 为通过行动 a 的访问次数, c 为探索常数, 控制探索与利用的权衡。式 (8) 中第一项 $Q(n, a)$ 代表利用已知高回报行动的倾向, 第二项则鼓励访问次数较少的行动以促进探索。

然而, 纯粹的随机模拟在如五子棋这类需要精确计算的游戏中心效率低下, 因为随机走子很少能生成对评估当前局面有意义的终局结果。AlphaZero 的关键创新在于将深度神经网络与 MCTS 深度集成, 用神经网络的学习能力指导搜索, 同时用搜索的经验提升网络 [1], [2]。具体而言, AlphaZero 将经典 UCT 公式修改为 PUCT 公式:

$$a^* = \arg \max_{a \in \mathcal{A}(n)} \left\{ Q(n, a) + c_{\text{puct}} \cdot P(n, a) \cdot \frac{\sqrt{N(n)}}{1 + N(n, a)} \right\} \quad (9)$$

其中 $P(n, a)$ 是由策略网络 $\pi_\theta(s)$ 给出的先验概率, 表示在未经过搜索的情况下网络认为行动 a 的优劣程度。相比于式 (8), 式 (9) 用先验概率 $P(n, a)$ 取代了 $\sqrt{\ln N(n)/N(n, a)}$ 中的 $\sqrt{\ln N(n)}$ 项, 这意味着网络的知识直接引导了探索的方向: 即使某个行动 a 尚未被充分探索 ($N(n, a)$ 较小), 只要网络赋予其较高的先验概率 $P(n, a)$, 它仍有较大可能被选中。系数 c_{puct} 控制着先验知识对探索的影响程度。

在节点扩展阶段, 对于新遇到的状态 s , 系统会调用神经网络同时获得策略先验 $\mathbf{p} = \pi_\theta(s)$ 和价值估计 $v = V_\theta(s)$ 。所有合法

行动 a 对应的先验概率 $P(s, a)$ 通过对 $\mathbf{p}$ 进行掩码处理 (将非法行动概率置零后重新归一化) 得到:

$$P(s, a) = \frac{\mathbf{p}_a \cdot \mathbb{I}[a \in \mathcal{A}_{\text{legal}}(s)]}{\sum_{b \in \mathcal{A}_{\text{legal}}(s)} \mathbf{p}_b} \quad (10)$$

该价值估计 v 将被用作该节点模拟结果的初始估计, 并参与后续的回传更新。

在 AlphaZero 及其变体中, 模拟阶段被极大简化甚至省略。经典 MCTS 的模拟阶段通常需要从扩展节点开始进行完整的随机走子直到终局, 以获得一个蒙特卡洛回报估计。而在 AlphaZero 框架下, 扩展节点时由价值网络给出的 v 直接作为该节点价值的估计, 无需进行耗时且可能不准确的随机模拟。这种方法大幅提升了搜索效率, 使得更多的计算资源可以用于探索更广的树分支。

回传阶段则按照访问路径更新所有祖先节点的统计信息。设一次搜索从根节点 n_0 出发, 经过 L 步到达叶节点 n_L , 并获得价值估计 v_{n_L} 。对于路径上的每一个节点 n_l ($l = 0, \dots, L-1$) 及其选择的行动 a_l , 更新公式为:

$$N(n_l) \leftarrow N(n_l) + 1 \quad (11)$$

$$N(n_l, a_l) \leftarrow N(n_l, a_l) + 1 \quad (12)$$

$$Q(n_l, a_l) \leftarrow Q(n_l, a_l) + \frac{v_{n_L} - Q(n_l, a_l)}{N(n_l, a_l)} \quad (13)$$

式 (13) 采用增量平均的方式更新行动价值, 确保 $Q(n_l, a_l)$ 始终是所有通过该行动获得的回报的平均值。如果对弈双方视角相反 (如黑方与白方), 则在回传时需要价值取反, 因为一方的高价值对应另一方的低价值。

完成预定次数的模拟后, 根节点 n_0 处各行动的访问次数 $N(n_0, a)$ 被用作改进策略 $\pi_{\text{MCTS}}(a|s_0)$ 的基础。通常通过温度参数 τ 对访问次数进行归一化处理:

$$\pi_{\text{MCTS}}(a|s_0; \tau) = \frac{N(n_0, a)^{1/\tau}}{\sum_b N(n_0, b)^{1/\tau}} \quad (14)$$

当 $\tau \rightarrow 0$ 时, π_{MCTS} 收敛到访问次数最多的行动 (确定性策略); 当 $\tau = 1$ 时, π_{MCTS} 正比于访问次数。在训练阶段的自对弈中, 通常采用 $\tau = 1$ 以保留探索性; 而在评估或与外部对手对弈时, 采用 $\tau \rightarrow 0$ 以获得最优策略。

神经网络的双目标训练则通过最小化以下复合损失函数实现:

$$\mathcal{L}(\theta) = (z - V_\theta(s))^2 - \pi_{\text{MCTS}}^\top \log \pi_\theta(s) + \lambda \|\theta\|^2 \quad (15)$$

其中 $z \in \{-1, 0, 1\}$ 为自对弈的实际终局结果 (从当前玩家视角), $V_\theta(s)$ 为价值网络的预测值, π_{MCTS} 为 MCTS 搜索得到的改进策略分布, $\pi_\theta(s)$ 为策略网络的预测分布, λ 为正则化系数。损失函数的第一项为价值损失 (均方误差), 驱使价值网络准确预测局面胜率; 第二项为策略损失 (交叉熵), 驱使策略网络逼近 MCTS 搜索得到的更优策略; 第三项为 L2 正则项, 防止过拟合。通过反向传播算法最小化该损失函数, 神经网络的参数 θ 得以更新, 从而将 MCTS 的搜索知识“蒸馏”到神经网络中。这一“搜索-评估-训练”的循环构成了自对弈强化学习的核心驱动力, 如图1所示。每一次迭代中, 当前神经网络参数 θ_t 用于指导 MCTS 生成高质量的对弈数据; 这些数据随后被用来训练新的网络参数 θ_{t+1} ; 更新后的网络在下一轮迭代中提供更准确的价值评估与策略先验, 进

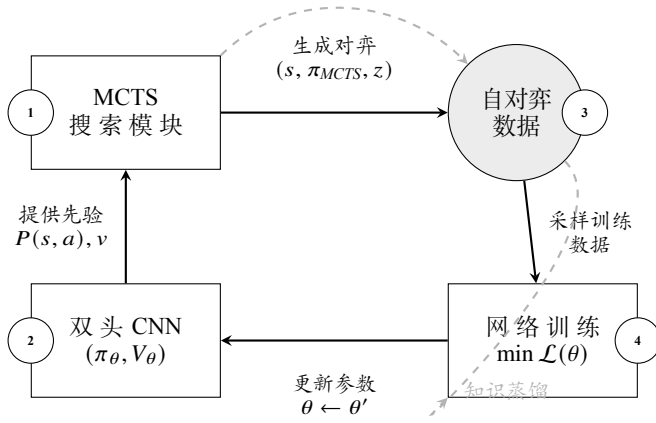
迭代循环 $t = 0, 1, \dots$ 

图 1: **GeepSeek** 自对弈训练循环示意图。该循环包含四个核心步骤: (1) MCTS 利用当前神经网络的先验知识和价值估计进行搜索; (2) 神经网络为 MCTS 提供先验概率 $P(s, a)$ 和价值估计 v ; (3) MCTS 生成的自对弈数据被收集存储; (4) 从数据中采样训练神经网络, 最小化损失函数 $\mathcal{L}(\theta)$ 。循环迭代进行, 实现策略的持续改进。

而引导更高效的搜索。理论上, 这一过程可收敛至纳什均衡策略 [1]。**GeepSeek** 系统正是基于这一理论框架, 通过精心设计的轻量化网络结构与高效的 MCTS 实现, 探索在资源受限条件下这一循环的有效性。

D. 深度学习基础与卷积神经网络架构

深度学习的兴起为复杂博弈问题的函数逼近提供了强有力的工具。在五子棋 AI 的语境下, 深度神经网络的核心任务在于学习从高维棋盘状态空间到策略分布与价值评估的有效映射。卷积神经网络因其在处理网格状数据 (如图像、棋盘) 时展现出的参数共享、平移不变性等优良特性, 成为博弈 AI 的首选架构 [9]。

对于五子棋状态 $s \in \mathbb{R}^{C \times H \times W}$ (其中 C 为通道数, $H = W = 15$ 为棋盘高宽), 一个典型的卷积层操作可表述为:

$$\mathbf{Z}^{(l)} = \sigma(\mathbf{W}^{(l)} * \mathbf{Z}^{(l-1)} + \mathbf{b}^{(l)}) \quad (16)$$

其中 $\mathbf{Z}^{(l)}$ 表示第 l 层的特征图, $\mathbf{W}^{(l)}$ 为卷积核权重, $*$ 表示卷积操作, $\mathbf{b}^{(l)}$ 为偏置项, $\sigma(\cdot)$ 为非线性激活函数 (如 ReLU)。卷积核的局部连接特性使得网络能够有效捕捉棋子间的局部模式, 例如活三、冲四等基本棋形, 而这些模式正是传统评估函数所依赖的关键特征。

然而, 随着网络深度增加, 梯度消失/爆炸问题会阻碍训练。残差学习通过引入快捷连接有效缓解了这一问题 [10]。残差块的基本结构可表示为:

$$\mathbf{Z}^{(l+1)} = \sigma(\mathcal{F}(\mathbf{Z}^{(l)}; \{\mathbf{W}\}) + \mathbf{Z}^{(l)}) \quad (17)$$

其中 $\mathcal{F}(\mathbf{Z}^{(l)}; \{\mathbf{W}\})$ 代表由若干卷积层组成的残差映射。这种设计使得网络能够轻松学习恒等映射, 确保信息在深层网络中有效流

动。在 **GeepSeek** 的轻量化设计中, 我们采用了简化版的残差块结构以平衡性能与计算开销。

策略头与价值头的网络分岔设计是双头架构的关键。给定骨干网络提取的高级特征 $\mathbf{Z}_{\text{feat}} \in \mathbb{R}^{d \times H' \times W'}$, 策略头通过额外的卷积层将特征维度映射到与行动空间相对应的维度:

$$\mathbf{p}_{\text{raw}} = \text{Conv2D}_{\text{policy}}(\mathbf{Z}_{\text{feat}}) \quad (18)$$

其中 $\mathbf{p}_{\text{raw}} \in \mathbb{R}^{1 \times H \times W}$, 随后通过扁平化与 softmax 操作得到合法行动空间上的概率分布:

$$\pi_{\theta}(a|s) = \frac{\exp(p_{\text{raw},a}) \cdot \mathbb{I}[a \in \mathcal{A}_{\text{legal}}(s)]}{\sum_{a' \in \mathcal{A}_{\text{legal}}(s)} \exp(p_{\text{raw},a'})} \quad (19)$$

这一设计确保网络输出始终是合法行动上的有效概率分布。

价值头则通过全局池化将空间特征聚合为全局向量表示, 再经全连接层回归到标量价值估计:

$$\mathbf{h}_{\text{global}} = \text{GlobalPool}(\mathbf{Z}_{\text{feat}}) \quad (20)$$

$$\mathbf{h}_{\text{fc1}} = \sigma(\mathbf{W}_1 \mathbf{h}_{\text{global}} + \mathbf{b}_1) \quad (21)$$

$$V_{\theta}(s) = \tanh(\mathbf{w}_2^T \mathbf{h}_{\text{fc1}} + b_2) \quad (22)$$

其中 $\tanh$ 激活函数将输出限制在 $[-1, 1]$ 区间, 对应于从当前玩家视角的胜率估计 (-1 为必败, 1 为必胜)。

神经网络的优化依赖于反向传播算法与梯度下降的变体。对于由式 (15) 定义的复合损失函数, 参数的更新遵循:

$$\theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} \mathcal{L}(\theta_t) \quad (23)$$

其中 η_t 为学习率。在实践中, 我们采用 Adam 优化器 [11], 它结合了动量法与自适应学习率调整, 其更新规则为:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}(\theta_t) \quad (24)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L}(\theta_t))^2 \quad (25)$$

$$\hat{\mathbf{m}}_t = \mathbf{m}_t / (1 - \beta_1^t), \quad \hat{\mathbf{v}}_t = \mathbf{v}_t / (1 - \beta_2^t) \quad (26)$$

$$\theta_{t+1} = \theta_t - \eta \cdot \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \varepsilon) \quad (27)$$

其中 $\beta_1, \beta_2 \in [0, 1)$ 为衰减率, ε 为数值稳定性常数。Adam 优化器在深度神经网络训练中表现出良好的收敛性与鲁棒性。

为了防止过拟合, 除了式 (15) 中的 L2 正则化外, 我们还在训练过程中采用了数据增强技术。得益于五子棋棋盘在旋转与反射下的对称性, 对于任意训练样本 (s, π, z) , 可以生成一组等价的增强样本:

$$\mathcal{T}(s, \pi, z) = \{(T_i(s), T_i(\pi), z) \mid T_i \in \mathbf{D}_4\} \quad (28)$$

其中 $\mathbf{D}_4$ 表示正方形的二面体群 (包含 4 个旋转与 4 个反射共 8 种对称变换), $T_i(\pi)$ 表示对策略概率分布进行相应的坐标变换。这一技术在不增加真实自对弈计算开销的前提下, 将训练数据量有效扩大了 8 倍, 显著提升了样本效率与模型泛化能力。

此外, 批归一化技术被应用于卷积层之后, 以加速训练并提高稳定性 [12]。对于批处理数据 $\mathcal{B} = \{\mathbf{x}_1, \dots, \mathbf{x}_m\}$, 批归一化操

作如下:

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m \mathbf{x}_i \quad (29)$$

$$\sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^m (\mathbf{x}_i - \mu_{\mathcal{B}})^2 \quad (30)$$

$$\hat{\mathbf{x}}_i = \frac{\mathbf{x}_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}} \quad (31)$$

$$\mathbf{y}_i = \gamma \hat{\mathbf{x}}_i + \beta \quad (32)$$

其中 γ, β 为可学习的缩放与平移参数, ε 为数值稳定性常数。批归一化确保了各层输入的分布稳定性, 缓解了内部协变量偏移问题。

综上所述, **GeepSeek** 系统所采用的轻量化 CNN 架构, 结合了现代深度学习的多项关键技术: 残差连接促进深层网络训练、双头设计实现策略与价值的联合学习、自适应优化器加速收敛、数据增强提升泛化、批归一化稳定训练过程。这些技术的协同作用, 使得在有限计算资源下实现有效的自对弈学习成为可能。

E. 自对弈强化学习框架的理论分析

自对弈强化学习作为 AlphaZero 系列算法的核心训练范式, 其理论本质是一种特殊的策略迭代过程, 其中智能体通过与自身副本的对弈来生成训练数据并优化策略。这一框架可形式化为一个双重优化问题: 内层优化是给定当前策略参数 θ_t 下, 通过 MCTS 搜索获得改进策略 π_{MCTS} ; 外层优化则是更新神经网络参数 θ 以最小化当前策略与改进策略之间的差异。

从动态系统视角看, 自对弈过程定义了一个在策略空间上的映射 $\Phi: \Theta \rightarrow \Theta$, 其中 Θ 为策略参数空间。一次完整的训练迭代包含以下复合映射:

$$\theta_{t+1} = \Phi(\theta_t) = \mathcal{U} \circ \mathcal{S} \circ \mathcal{E}(\theta_t) \quad (33)$$

其中: $-\mathcal{E}: \theta \mapsto \mathcal{D}$ 为经验生成映射, 表示以策略 π_{θ} 进行自对弈生成数据集 $\mathcal{D}$; $-\mathcal{S}: \mathcal{D} \mapsto \mathcal{D}'$ 为搜索改进映射, 表示对 $\mathcal{D}$ 中的每个状态应用 MCTS 得到改进策略标签; $-\mathcal{U}: \mathcal{D}' \mapsto \theta'$ 为参数更新映射, 表示基于改进后的数据集更新网络参数。

这种训练机制的理论收敛性依赖于几个关键性质。首先, MCTS 搜索可视为一种策略提升算子, 满足策略改进定理的条件。在理想情况下, 给定准确的价值函数 V^{π} , 通过单步前瞻搜索得到的贪婪策略 π' 满足:

$$V^{\pi'}(s) \geq V^{\pi}(s) \quad \forall s \in \mathcal{S} \quad (34)$$

在实际中, 由于 MCTS 使用的价值网络 V_{θ} 存在近似误差, 且搜索深度有限, 式 (34) 的成立需要一定的条件保证。

其次, 自对弈数据的分布动态变化引入了非平稳性挑战。训练数据并非来自固定的状态分布, 而是随着策略 π_{θ_t} 的演化而不断变化。这种非平稳性可能导致训练不稳定, 理论上可通过重要性采样或经验回放技术部分缓解 [13]。**GeepSeek** 采用类似于 AlphaZero 的经验缓冲区设计, 但受限于计算资源, 缓冲区规模相对较小, 这要求网络具备良好的泛化能力以应对分布偏移。

从博弈论视角分析, 自对弈过程可视为求解两人零和博弈纳什均衡的迭代方法。著名的虚拟对弈算法及其现代变体与自对弈

强化学习有深刻的联系 [14]。在虚拟对弈中, 每个玩家基于对手的历史平均策略选择最优反应, 最终收敛至纳什均衡。自对弈强化学习可视为一种广义的虚拟对弈, 其中 MCTS 充当了近似最优反应算子, 而神经网络的训练则实现了策略的平均化。

自对弈训练的 efficiency 在很大程度上取决于探索策略的设计。纯粹的贪婪策略改进容易陷入局部最优, 为此需要在训练过程中引入系统的探索机制。AlphaZero 采用 Dirichlet 噪声注入的方式, 在根节点的先验概率上添加噪声:

$$P'(s, a) = (1 - \varepsilon) \cdot P(s, a) + \varepsilon \cdot \eta_a, \quad \eta \sim \text{Dir}(\alpha) \quad (35)$$

其中 ε 控制噪声强度, α 为 Dirichlet 分布的浓度参数。这种探索机制在保证主要探索高先验概率行动的同时, 以一定概率尝试低概率但可能带来突破的行动, 有助于避免策略过早收敛到次优解。

温度调度是另一个重要的探索控制机制。如式 (14) 所示, 温度参数 τ 调节了策略的确定性程度。在训练初期, 采用较高的温度 ($\tau = 1$) 鼓励探索; 随着训练进行, 逐渐降低温度以聚焦于当前最优策略。温度的衰减策略通常遵循预定义的调度表或根据训练进度自适应调整。

从样本复杂度的角度分析, 自对弈强化学习属于一种在策略学习方法, 其数据效率相对较低。每个训练样本仅使用一次或少数几次, 这与使用大规模静态数据集的监督学习形成对比。提高样本效率的方法包括: 1) 增加每次自对弈生成的数据量 (通过更深的搜索); 2) 使用经验回放重复利用旧数据; 3) 应用数据增强技术如式 (28) 所示。**GeepSeek** 在资源约束下主要依赖第三种方法。

最后, 训练过程的评估与监控需要合适的度量指标。除了常见的损失函数值外, 自对弈强化学习中以下几个指标尤为重要:

$$\text{自对弈胜率} = \mathbb{P}_{(s, \pi, z) \sim \mathcal{D}_{\text{selfplay}}} [z = 1] \quad (36)$$

$$\text{策略熵} = -\mathbb{E}_{s \sim \mathcal{D}} \left[\sum_a \pi_{\theta}(a|s) \log \pi_{\theta}(a|s) \right] \quad (37)$$

$$\text{价值准确性} = 1 - \mathbb{E}_{(s, z) \sim \mathcal{D}} [|z - V_{\theta}(s)|] \quad (38)$$

策略熵反映了策略的确定性程度, 在训练初期应保持较高值以确保探索, 随后逐渐降低。价值准确性则衡量了价值网络的预测能力, 其提升通常意味着网络对局面判断能力的增强。

综上所述, 自对弈强化学习框架融合了策略迭代、博弈论优化与深度函数逼近的理论要素。其在五子棋这一具体领域中的应用, 既是对这些理论的实证检验, 也为进一步的理论分析提供了丰富的实验场景。**GeepSeek** 系统的设计充分考虑了这些理论要素在资源受限条件下的可实现性, 力求在有限的计算预算内最大化学习效率。

III. **GeepSeek** 系统设计与实现

A. 整体架构设计

GeepSeek 系统的整体架构遵循自对弈强化学习的基本范式, 但在各组件设计上针对有限计算资源进行了精心优化。如图2所示, 系统主要由四个核心模块组成: 自对弈数据生成模块、神经网络模型模块、训练优化模块以及评估对弈模块。这些模块通过

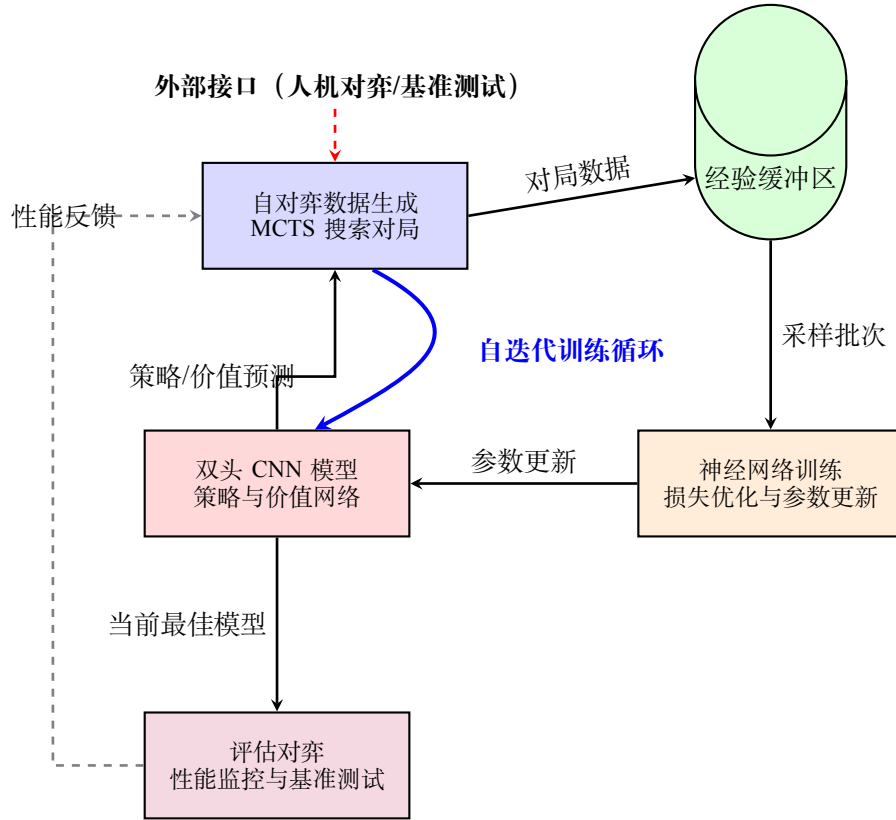


图 2: **GeepSeek** 系统整体架构图。系统包含四个核心模块：自对弈数据生成、经验缓冲区、神经网络训练和评估对弈，形成闭环的自迭代训练循环。

协同工作，实现了“生成数据-训练模型-评估性能”的完整迭代循环。

系统的工作流程始于自对弈数据生成模块。该模块使用当前神经网络模型 θ_t 作为策略先验，结合蒙特卡洛树搜索进行完整的对弈模拟。每局对弈从空棋盘开始，双方交替行动，每一步都通过 MCTS 搜索决定。对弈过程中生成的状态-策略-价值三元组 $(s, \pi_{\text{MCTS}}, z)$ 被记录并存入经验缓冲区。为了在有限计算资源下最大化数据多样性，我们实现了并行自对弈机制，可在单机上同时运行多个对弈进程。

经验缓冲区采用有限容量的先进先出队列设计。设缓冲区容量为 B ，当新数据加入导致缓冲区溢出时，最早的数据将被移除。这种设计确保了训练数据始终来自相对较近的策略版本，减少了因策略分布漂移过大而导致的训练不稳定。在实际实现中，我们设置 $B = 10^5$ 个样本，这一规模在单机内存可承受范围内，同时能够提供足够的样本多样性。

神经网络训练模块从缓冲区中随机采样小批量数据，通过最小化式 (15) 定义的复合损失函数来更新模型参数。训练过程采用带权重衰减的 Adam 优化器，学习率遵循余弦退火调度。为了加速收敛并提高泛化能力，我们在训练中应用了棋盘对称性数据增强，如式 (28) 所示。

评估对弈模块负责监控训练进度和模型性能。该模块定期（如每完成 100 次训练迭代）组织评估比赛，让当前模型与固定基准对手（如先前版本的模型、传统 $\alpha-\beta$ 搜索算法等）进行多局对弈，统计胜率并计算 Elo 等级分变化。评估结果作为是否更新“当前

最佳模型”的依据，并可用于调整训练超参数。

系统的一个关键设计特点是组件间的松耦合。各模块通过清晰的接口定义进行交互，使得单个模块的改进或替换（如尝试不同的网络架构、优化算法等）无需重写整个系统。这种模块化设计不仅提高了代码的可维护性，也为后续研究提供了灵活的扩展基础。

在实现层面，**GeepSeek** 采用 Python 作为主要编程语言，利用 PyTorch 深度学习框架实现神经网络组件。蒙特卡洛树搜索算法通过 C++ 扩展实现关键性能路径，并通过 Python 接口进行调用，以平衡开发效率与运行时性能。系统支持 CPU 和 GPU 混合计算模式：MCTS 搜索主要在 CPU 上进行，而神经网络的前向传播和反向传播则在 GPU 上执行。这种异构计算策略充分利用了不同硬件的优势，在资源受限条件下实现了相对高效的计算。

B. 双头卷积神经网络架构设计

GeepSeek 系统的核心组件是一个经过精心设计的轻量化双头卷积神经网络 (Dual-Head CNN)，该网络旨在以最小的计算开销实现策略预测与价值评估的双重功能。网络采用共享特征提取层与分离输出头的架构，实现了参数效率与功能完备性的平衡。

1) 输入表示与特征编码：网络输入为五子棋棋盘状态的张量表示 $\mathbf{S} \in \mathbb{R}^{4 \times 15 \times 15}$ 。这一四通道设计相较于传统的单通道（黑

白棋子)表示,包含了更丰富的游戏信息:

$$\begin{aligned} \mathbf{S}[0, :, :] &= \mathbb{I}_{\text{黑棋}} && (\text{黑方棋子位置}) \\ \mathbf{S}[1, :, :] &= \mathbb{I}_{\text{白棋}} && (\text{白方棋子位置}) \\ \mathbf{S}[2, :, :] &= \mathbb{I}_{\text{空位}} && (\text{合法落子位置}) \\ \mathbf{S}[3, :, :] &= t/225 && (\text{回合数归一化}) \end{aligned} \quad (39)$$

其中 $\mathbb{I}_{\text{类别}}$ 为指示函数, t 为当前回合数 (从 0 开始)。回合数特征的引入使网络能够感知游戏阶段,这在终局精确计算与开局策略选择中尤为重要。输入张量首先经过一个归一化层,将各通道值缩放到 $[0, 1]$ 区间。

特征提取部分由一个初始卷积层和四个残差块组成。初始卷积层使用 64 个 3×3 卷积核,步长为 1,填充为 1,保持空间分辨率不变:

$$\mathbf{F}_0 = \text{ReLU} \left(\text{BN} \left(\text{Conv}_{3 \times 3}^{64}(\mathbf{S}) \right) \right) \in \mathbb{R}^{64 \times 15 \times 15} \quad (40)$$

其中 BN 表示批归一化操作。这一设计确保了即使经过多层变换,特征图的空间维度仍与原始棋盘保持一致,为后续的位置敏感预测奠定基础。

2) 轻量化残差设计: 考虑到计算资源的限制, **GeepSeek** 采用了简化的残差块结构,而非原始 ResNet 中的标准残差块。每个残差块包含两个 3×3 卷积层,通道数保持为 64,中间加入批归一化与 ReLU 激活:

$$\begin{aligned} \mathbf{F}_{\text{mid}} &= \text{ReLU} \left(\text{BN} \left(\text{Conv}_{3 \times 3}^{64}(\mathbf{F}_{\text{in}}) \right) \right) \\ \mathbf{F}_{\text{res}} &= \text{BN} \left(\text{Conv}_{3 \times 3}^{64}(\mathbf{F}_{\text{mid}}) \right) \\ \mathbf{F}_{\text{out}} &= \text{ReLU}(\mathbf{F}_{\text{res}} + \mathbf{F}_{\text{in}}) \end{aligned} \quad (41)$$

这一“瓶颈”设计 (两个 3×3 卷积直接相连) 相较于标准的“瓶颈结构” ($1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$) 减少了约 33% 的参数与计算量。四个残差块顺序连接,形成了深度为 8 个卷积层的特征提取主干,总参数量仅为:

$$P_{\text{res}} = 4 \times (2 \times 3 \times 3 \times 64 \times 64) = 294,912 \text{ 参数} \quad (42)$$

批归一化层的使用不仅加速了训练收敛,还起到了轻微的正则化效果。ReLU 激活函数的选择基于其计算简单且在实际应用中表现良好的特性。尽管近年出现了如 Swish、GELU 等更复杂的激活函数,但在我们的实验中发现,对于五子棋这类相对小规模的问题,ReLU 已能提供足够的表达能力。

3) 策略头设计: 策略头负责从共享特征 $\mathbf{F}_{\text{shared}} \in \mathbb{R}^{64 \times 15 \times 15}$ 中预测每个合法落子位置的概率分布。首先,一个 1×1 卷积层将通道数从 64 降至 2:

$$\mathbf{P}_{\text{raw}} = \text{Conv}_{1 \times 1}^2(\mathbf{F}_{\text{shared}}) \in \mathbb{R}^{2 \times 15 \times 15} \quad (43)$$

这一设计基于以下观察:对于落子预测,仅需区分当前位置是否为“好着法”,而不需要复杂的多通道特征表示。两个通道分别对应“正类”与“负类”的原始分数。

随后,特征图被展平为 450 维向量,并通过一个全连接层映射到 225 维输出空间,对应棋盘上所有可能位置:

$$\mathbf{p}_{\text{logits}} = \mathbf{W}_{\text{policy}} \cdot \text{Flatten}(\mathbf{P}_{\text{raw}}) + \mathbf{b}_{\text{policy}} \in \mathbb{R}^{225} \quad (44)$$

最后,应用掩码操作与 softmax 函数生成合法的概率分布:

$$\pi_{\theta}(a|s) = \frac{\exp(p_{\text{logits},a}) \cdot \mathbb{I}[a \in \mathcal{A}_{\text{legal}}(s)]}{\sum_{a' \in \mathcal{A}_{\text{legal}}(s)} \exp(p_{\text{logits},a'})} \quad (45)$$

这种“先全连接后掩码”的设计允许网络为所有位置生成预测,但仅对合法位置进行归一化,确保了输出分布的合理性。

4) 价值头设计: 价值头的目标是评估当前局面 s 对当前玩家的胜率,输出值 $V_{\theta}(s) \in [-1, 1]$,其中 1 表示必胜, -1 表示必败。与策略头不同,价值评估需要全局信息而非局部细节。因此,我们首先应用全局平均池化 (GAP) 来聚合空间信息:

$$\mathbf{h}_{\text{global}} = \text{GAP} \left(\text{Conv}_{1 \times 1}^{32}(\mathbf{F}_{\text{shared}}) \right) \in \mathbb{R}^{32} \quad (46)$$

全局平均池化计算每个通道上所有空间位置的平均值,产生对平移不变的全局特征表示。

随后,两个全连接层逐步压缩特征维度:

$$\begin{aligned} \mathbf{h}_1 &= \text{ReLU}(\mathbf{W}_1 \mathbf{h}_{\text{global}} + \mathbf{b}_1) \in \mathbb{R}^{64} \\ \mathbf{h}_2 &= \mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2 \in \mathbb{R}^1 \\ V_{\theta}(s) &= \tanh(\mathbf{h}_2) \end{aligned} \quad (47)$$

tanh 激活函数确保输出值落在 $[-1, 1]$ 区间,这与博弈结果的取值范围 $(-1, 0, 1)$ 自然对齐。值得注意的是,价值头不包含批归一化层,因为经验表明在最后的回归层中使用批归一化会降低预测的稳定性。

5) 参数效率分析: **GeepSeek** 的网络设计在参数效率方面进行了精心优化。网络总参数量计算如下:

$$\begin{aligned} P_{\text{total}} &= P_{\text{initial}} + P_{\text{res}} + P_{\text{policy}} + P_{\text{value}} \\ &= (3 \times 3 \times 4 \times 64) + 294,912 + (1 \times 1 \times 64 \times 2 + 450 \times 225) \\ &\quad + (1 \times 1 \times 64 \times 32 + 32 \times 64 + 64 \times 1) \\ &\approx 2304 + 294,912 + 128 + 101,250 + 2048 + 2048 + 64 \\ &\approx 400,754 \text{ 参数} \end{aligned} \quad (48)$$

这一规模仅约为 AlphaZero 原始网络 (约 500 万参数) 的 8%,甚至小于一些现代的轻量化模型如 MobileNetV2 的最小配置。参数分布分析显示,策略头全连接层占据了约 25% 的参数 (101,250),这是必要的代价以确保对 225 个位置的细粒度预测能力。

在计算复杂度方面,网络一次前向传播的浮点运算次数 (FLOPs) 约为:

$$\text{FLOPs} \approx 2 \times P_{\text{total}} \times \text{batch_size} \approx 0.8 \text{ MFLOPs} \quad (\text{batch_size} = 1) \quad (49)$$

这使得在普通 CPU 上也能达到每秒数百次的前向传播速度,为 MCTS 的高效搜索提供了基础。

6) 训练稳定性增强技术: 为在有限的训练数据下确保网络训练的稳定性,我们采用了以下技术:

1. 梯度裁剪: 在反向传播过程中,将梯度范数限制在阈值 $\tau = 5.0$ 以内:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min \left(1, \frac{\tau}{\|\mathbf{g}\|_2} \right) \quad (50)$$

这防止了梯度爆炸问题,特别在训练初期网络预测不准确时尤为重要。

算法 1 神经网络引导的蒙特卡洛树搜索 (Neural MCTS)**Input:** 初始状态 s_0 , 神经网络参数 θ , 模拟次数 N_{sim} , 探索常数 c_{puct} **Output:** 改进策略分布 $\pi_{\text{MCTS}} \in \Delta(\mathcal{A})$

```

1 初始化根节点  $n_0$ , 状态为  $s_0$   $\pi_{\text{MCTS}} \leftarrow \mathbf{0}^{|\mathcal{A}|}$ ; // 初始化策略向量
2 for  $i = 1$  to  $N_{\text{sim}}$  do
3    $n \leftarrow n_0, s \leftarrow s_0$ ; // 从根节点开始
4    $\text{path} \leftarrow []$ ; // 记录访问路径
5   while  $n$  不是叶节点且  $s \notin \mathcal{S}_T$  do
6     计算所有行动  $a \in \mathcal{A}(s)$  的 UCB 分数  $\text{UCB}(a) = Q(n, a) + c_{\text{puct}} \cdot P(n, a) \cdot \frac{\sqrt{N(n)}}{1+N(n, a)}$  选择行动  $a^* = \arg \max_a \text{UCB}(a)$ 
7     将  $(n, a^*)$  加入  $\text{path}$   $s \leftarrow f(s, a^*)$ ; // 状态转移
8      $n \leftarrow \text{Child}(n, a^*)$ ; // 转移到子节点
9   end
10  if  $s \notin \mathcal{S}_T$  then // 网络前向传播
11     $(\mathbf{p}, v) \leftarrow \text{NeuralNetwork}_{\theta}(s)$ ;
12    for  $a \in \mathcal{A}(s)$  do
13       $P(n, a) \leftarrow \mathbf{p}_a \cdot \mathbb{I}[a \in \mathcal{A}_{\text{legal}}(s)]$  初始化  $N(n, a) \leftarrow 0, Q(n, a) \leftarrow 0$ 
14    end
15  else // 从当前玩家视角
16     $v \leftarrow \text{GameResult}(s)$ ;
17  end
18  for  $(n, a)$  in  $\text{path}$  do // 增量更新
19     $N(n) \leftarrow N(n) + 1$   $N(n, a) \leftarrow N(n, a) + 1$   $Q(n, a) \leftarrow Q(n, a) + \frac{v - Q(n, a)}{N(n, a)}$ ;
20     $v \leftarrow -v$ ; // 视角切换
21  end
22  for  $a \in \mathcal{A}(s_0)$  do
23     $\pi_{\text{MCTS}}[a] \leftarrow N(n_0, a)$ 
24  end
25  $\pi_{\text{MCTS}} \leftarrow \pi_{\text{MCTS}} / \sum_a \pi_{\text{MCTS}}[a]$ ; // 归一化
26 return  $\pi_{\text{MCTS}}$ 

```

2. 学习率热身: 在前 $W = 1000$ 个训练步中使用线性学习率热身策略:

$$\eta_t = \eta_{\text{base}} \cdot \frac{t}{W}, \quad t < W \quad (51)$$

这允许网络在训练初期缓慢适应数据分布, 避免因初始学习率过高导致的震荡。

3. 标签平滑: 在策略头的训练中, 对 MCTS 提供的目标分布 π_{MCTS} 进行轻微平滑:

$$\pi_{\text{target}} = (1 - \epsilon) \cdot \pi_{\text{MCTS}} + \epsilon \cdot \mathcal{U}(\mathcal{A}_{\text{legal}}) \quad (52)$$

其中 $\epsilon = 0.1$, $\mathcal{U}$ 为均匀分布。这降低了网络对可能含噪的 MCTS 分布的过度自信。

这些设计的协同作用使得 **GeepSeek** 的双头 CNN 能够在有限的训练数据与计算资源下, 稳定地学习到有效的策略与价值函数, 为后续的自对弈迭代提供了可靠的基础组件。

C. 蒙特卡洛树搜索算法实现细节

GeepSeek 系统中的蒙特卡洛树搜索 (MCTS) 算法是连接神经网络与自对弈训练的关键桥梁。算法 1 详细描述了结合神经网络引导的 MCTS 完整流程, 该算法实现了在有限计算预算下的高效策略搜索。

1) 节点数据结构优化: 为实现高效的树搜索, 我们设计了紧凑的节点数据结构。每个节点 n 存储以下信息:

state: 棋盘状态 (仅在必要时存储)

$N(n)$: 节点总访问次数 (32 位整数)

$Q(n)$: 各行动价值估计 (32 位浮点数组)

$P(n)$: 先验概率分布 (16 位浮点数组)

$N(n, a)$: 行动访问次数 (16 位整数数组) (53)

这种混合精度设计基于以下观察: 访问次数通常不会超过 $2^{16} = 65536$, 而价值估计需要更高精度。节点采用延迟状态存储策略, 仅当节点被扩展时才存储完整棋盘状态, 否则仅存储状态哈希值以减少内存占用。

树节点的内存占用可估算为:

$$M_{\text{node}} = \underbrace{4}_{N(n)} + \underbrace{4|\mathcal{A}|}_{Q} + \underbrace{2|\mathcal{A}|}_{P} + \underbrace{2|\mathcal{A}|}_{N} \approx 8|\mathcal{A}| + 4 \text{ 字节} \quad (54)$$

对于五子棋, $|\mathcal{A}| \leq 225$, 单个节点内存约 1.8KB。一次 $N_{\text{sim}} = 800$ 的搜索通常生成约 2000 个节点, 总内存占用约 3.6MB, 在普通计算机上完全可行。

2) 虚拟损失与并行搜索: 为充分利用多核 CPU, **GeepSeek** 实现了并行 MCTS 算法。多个搜索线程同时从根节点出发进行模拟, 这引入了线程间的竞争条件。我们采用虚拟损失机制解决此问题: 当线程选择节点 n 并执行行动 a 时, 立即为该行动增加虚拟损失 δ_v :

$$Q_{\text{virtual}}(n, a) = Q(n, a) - \delta_v \quad (55)$$

其中 $\delta_v = 1.0$ 。这暂时降低了该行动的吸引力, 促使其他线程探索不同分支, 增加了搜索的多样性。当线程完成模拟并回传真实价值 v 时, 虚拟损失被抵消:

$$Q(n, a) \leftarrow Q(n, a) + \frac{v + \delta_v - Q(n, a)}{N(n, a)} \quad (56)$$

3) 探索增强技术: 为在自对弈训练早期促进探索, 我们在根节点添加 Dirichlet 噪声 [15]。具体而言, 对于根节点 n_0 的先验概率 $\mathbf{P}(n_0)$, 我们按以下方式添加噪声:

$$\begin{aligned} \eta &\sim \text{Dirichlet}(\alpha \cdot \mathbf{1}_{|\mathcal{A}|}) \\ \mathbf{P}'(n_0) &= (1 - \epsilon) \cdot \mathbf{P}(n_0) + \epsilon \cdot \eta \end{aligned} \quad (57)$$

其中 $\alpha = 0.3$ 控制噪声集中程度, $\epsilon = 0.25$ 控制噪声强度。这种噪声注入仅在自对弈的前 30 步中应用, 确保了早期探索的充分性。

此外, 我们实现了渐进式剪枝策略。对于访问次数极低的行动 ($N(n, a) < N_{\min}$, 其中 $N_{\min} = 3$), 在后续选择中将其 UCB 分数乘以衰减因子 $\beta = 0.5$:

$$\text{UCB}_{\text{pruned}}(a) = \begin{cases} \text{UCB}(a) \cdot \beta, & \text{if } N(n, a) < N_{\min} \\ \text{UCB}(a), & \text{otherwise} \end{cases} \quad (58)$$

这种软剪枝策略既避免了完全丢弃潜在有价值的行动, 又引导搜索资源集中于更有希望的分支。

4) 终局精确求解优化: 对于接近终局的局面, MCTS 的随机性可能导致不必要的搜索浪费。**GeepSeek** 集成了一个轻量化的终局求解器, 当检测到以下条件之一时触发:

1. 剩余空位数 $\leq T_{\text{solve}} = 12$
2. 存在直接的胜利威胁 (活四或双冲四)
3. 搜索深度 $\geq D_{\max} = 8$ 且价值估计 $|v| > 0.95$

终局求解器采用带记忆化的 α - β 搜索, 深度限制为剩余空位数。求解结果直接覆盖 MCTS 的价值估计 v , 显著提高了终局决策的准确性。

算法 1 的时间复杂度主要受模拟次数 N_{sim} 和平均模拟深度 L_{avg} 影响。每次模拟包含三个阶段: 选择阶段 $O(L_{\text{avg}})$, 扩展与评估阶段 $O(1)$ (神经网络前向传播为常数时间), 回传阶段 $O(L_{\text{avg}})$ 。总时间复杂度为:

$$T_{\text{MCTS}} = O(N_{\text{sim}} \cdot (2L_{\text{avg}} + C_{\text{NN}})) \quad (59)$$

其中 C_{NN} 为神经网络前向传播的常数开销。在 **GeepSeek** 的默认配置下 ($N_{\text{sim}} = 800$, $L_{\text{avg}} \approx 30$), 单次搜索耗时约 50-80ms, 满足实时对弈的需求。

5) 温度调度策略: 如式 (14) 所述, 温度参数 τ 控制着从访问次数到策略分布的转换确定性。**GeepSeek** 采用自适应的温

度调度策略:

$$\tau(t) = \max\left(0.1, \exp\left(-\frac{t}{T_{\text{decay}}}\right)\right) \quad (60)$$

其中 t 为训练迭代次数, $T_{\text{decay}} = 1000$ 为衰减常数。在训练初期 ($t < 100$), $\tau \approx 1.0$ 鼓励充分探索; 随着训练进行, τ 逐渐降低, 最终稳定在 0.1 以产生近乎确定性的策略。在评估阶段, 固定使用 $\tau = 0.01$ 以确保最优性能。

这些优化技术的综合应用使得 **GeepSeek** 的 MCTS 模块在有限的计算资源下达到了较高的搜索效率, 为自对弈训练提供了高质量的策略改进信号。

D. 自对弈训练流程与优化策略

GeepSeek 系统的自对弈训练流程实现了算法 1 所描述的搜索过程与实际对弈的紧密结合, 通过迭代优化不断改进神经网络参数。算法 2 展示了完整的训练流程, 揭示了数据生成、网络训练与模型评估之间的相互作用关系。

1) 并行自对弈数据生成: 自对弈数据生成是训练流程中最耗时的部分。为实现高效的数据生成, **GeepSeek** 采用基于进程池的并行化策略。对于 G 局自对弈, 我们创建 P 个工作进程 (通常 $P = \min(G, N_{\text{cores}})$, 其中 N_{cores} 为 CPU 核心数)。每个工作进程独立执行以下操作:

环境初始化: 创建新的棋盘环境, 重置为初始状态 $s_0 \rightarrow$ **对局循环:** 按照算法 1 进行决策, 直至游戏终止 $\rightarrow$ **数据记录:** 记录每一步的状态 s_t 、MCTS 策略分布 π_t 和最终结果 z

为防止所有进程产生过于相似的对局, 我们在不同进程中采用不同的随机种子。此外, 为平衡探索与利用, 我们对前 $K_{\text{explore}} = 30$ 步添加 Dirichlet 噪声, 如式 (57) 所示。

单局对局的数据量取决于对局长度 L 。对于五子棋, L 通常在 30 到 60 之间。每局生成的数据样本数为 L , 每个样本包含:

$$\begin{aligned} \text{状态: } &\mathbb{R}^{4 \times 15 \times 15} \quad (4 \times 225 = 900 \text{ 个浮点数}) \\ \text{策略: } &\mathbb{R}^{225} \quad (225 \text{ 个浮点数}) \\ \text{价值: } &\mathbb{R} \quad (1 \text{ 个浮点数}) \end{aligned} \quad (61)$$

因此, 一局对局约产生 $L \times (900 + 225 + 1) \approx 40 \times 1126 = 45,040$ 个浮点数, 即约 180KB 数据 (单精度浮点数)。默认配置下, 每轮训练生成 $G = 100$ 局对局, 数据量约 18MB。

2) 经验缓冲区设计与采样策略: 经验缓冲区 $\mathcal{D}$ 采用环形缓冲区实现, 容量 $B_{\max} = 100,000$ 个样本。缓冲区维护两个指针: 写指针 p_w (指向下一个写入位置) 和读指针 p_r (指向下一个读取位置)。当写入新数据导致缓冲区溢出时, 最旧的数据被覆盖。

采样策略采用分层抽样与优先级回放相结合的方法。首先, 根据对局结果将样本分为三类:

$$\mathcal{D} = \mathcal{D}_{\text{win}} \cup \mathcal{D}_{\text{lose}} \cup \mathcal{D}_{\text{draw}} \quad (62)$$

其中 $\mathcal{D}_{\text{win}}$ 包含获胜方的所有中间状态, $\mathcal{D}_{\text{lose}}$ 包含失败方的状态, $\mathcal{D}_{\text{draw}}$ 包含平局状态。采样时, 按比例 0.4 : 0.4 : 0.2 从三个子集中抽取, 确保正负样本的平衡。

对于每个样本 (s, π, z) , 我们定义优先级 p 为:

$$p = \alpha \cdot |z - V_{\theta}(s)| + (1 - \alpha) \cdot \text{KL}(\pi \| \pi_{\theta}(s)) \quad (63)$$

算法 2 GeepSeek 自对弈训练主循环**Input:** 初始网络参数 θ_0 , 总迭代次数 T , 自对弈局数 G , MCTS 模拟次数 N_{sim} **Output:** 训练后的网络参数 θ_T

```

27  $\theta_{\text{best}} \leftarrow \theta_0$  ; // 记录最佳参数
28  $\mathcal{D} \leftarrow \emptyset$  ; // 初始化经验缓冲区
29 for  $t = 0$  to  $T - 1$  do
    // 阶段 1: 并行自对弈生成数据
30  $\mathcal{D}_{\text{new}} \leftarrow \emptyset$  for  $g = 1$  to  $G$  do in parallel // 并行执行  $G$  局自对弈
    | ;
31 | 使用当前网络  $\theta_t$  进行自对弈 应用算法1进行每步决策 记录对局轨迹  $\tau = [(s_i, \pi_i, z)]_{i=1}^L$   $\mathcal{D}_{\text{new}}.\text{add}(\tau)$ 
32 end
    // 阶段 2: 经验缓冲区更新
33  $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_{\text{new}}$  if  $|\mathcal{D}| > B_{\text{max}}$  then
34 | 移除最旧的  $|\mathcal{D}| - B_{\text{max}}$  个样本
35 end
    // 阶段 3: 神经网络训练
36 for  $e = 1$  to  $E_{\text{epoch}}$  do
37 | 从  $\mathcal{D}$  中随机采样批次  $\mathcal{B}$  计算损失:  $\mathcal{L} = \mathcal{L}_{\text{value}} + \mathcal{L}_{\text{policy}} + \lambda \mathcal{L}_{\text{reg}}$  更新参数:  $\theta_t \leftarrow \theta_t - \eta_t \nabla_{\theta} \mathcal{L}$ 
38 end
    // 阶段 4: 模型评估与选择
39 if  $t \bmod F_{\text{eval}} = 0$  then
40 |  $w_{\text{rate}} \leftarrow \text{Evaluate}(\theta_t, \theta_{\text{best}})$  if  $w_{\text{rate}} > \tau_{\text{accept}}$  then
41 | |  $\theta_{\text{best}} \leftarrow \theta_t$ 
42 | end
43 end
44  $\theta_{t+1} \leftarrow \theta_t$ 
45 end
46 return  $\theta_{\text{best}}$ 

```

其中 $\alpha = 0.7$ 控制价值误差与策略差异的权重。高优先级的样本有更高概率被采样, 这加速了对困难样本的学习。

3) 损失函数设计与梯度优化: 训练损失函数如式 (15) 所定义, 由三部分组成。在实际实现中, 我们对各项损失进行了加权调整:

$$\begin{aligned}
 \mathcal{L}_{\text{total}} &= \lambda_v \mathcal{L}_{\text{value}} + \lambda_{\pi} \mathcal{L}_{\text{policy}} + \lambda_r \mathcal{L}_{\text{reg}} \\
 \mathcal{L}_{\text{value}} &= \frac{1}{B} \sum_{i=1}^B (z_i - V_{\theta}(s_i))^2 \\
 \mathcal{L}_{\text{policy}} &= -\frac{1}{B} \sum_{i=1}^B \pi_i^{\top} \log \pi_{\theta}(s_i) \\
 \mathcal{L}_{\text{reg}} &= \|\theta\|_2^2
 \end{aligned} \tag{64}$$

其中 $\lambda_v = 1.0$, $\lambda_{\pi} = 1.0$, $\lambda_r = 10^{-4}$ 。价值损失使用均方误差 (MSE), 策略损失使用交叉熵。

优化过程采用 AdamW 优化器 [11], [16], 结合余弦退火学习率调度:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{t}{T}\pi\right)\right) \tag{65}$$

其中 $\eta_{\max} = 0.001$, $\eta_{\min} = 0.0001$, T 为总训练步数。学习率热身在前 $W = 1000$ 步使用线性增长。

梯度裁剪被应用于所有参数更新:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{\tau}{\|\mathbf{g}\|_2}\right), \quad \tau = 5.0 \tag{66}$$

这防止了训练初期因梯度爆炸导致的不稳定。

4) 模型评估与选择策略: 模型评估是训练循环中的关键环节, 用于决定是否接受新训练的模型。评估过程如下:

1. **对手选择**: 当前模型 θ_t 与当前最佳模型 θ_{best} 进行 $N_{\text{eval}} = 100$ 局对弈 2. **对弈配置**: 双方均使用 MCTS 进行决策, 模拟次数 $N_{\text{sim}} = 800$, 温度 $\tau = 0.01$ 3. **结果统计**: 记录胜、负、平局数, 计算胜率 w_{rate} 4. **决策规则**: 如果 $w_{\text{rate}} > \tau_{\text{accept}} = 0.55$, 则接受新模型为最佳模型

这一评估机制确保了模型性能的单调改进 (以高概率)。为避免评估过程过于频繁影响训练效率, 评估间隔设为 $F_{\text{eval}} = 5$ 轮训练迭代。

除胜率外, 我们还监控以下训练指标:

$$\text{策略熵: } H(\pi) = -\frac{1}{B} \sum_{i=1}^B \sum_a \pi_i(a) \log \pi_i(a)$$

$$\text{价值准确率: } A_v = \frac{1}{B} \sum_{i=1}^B \mathbb{I}[|z_i - V_{\theta}(s_i)| < 0.5]$$

$$\text{梯度范数: } \|\nabla\| = \frac{1}{|\theta|} \sum_j \|\nabla_{\theta_j}\|_2 \tag{67}$$

这些指标提供了训练过程的细粒度监控，有助于早期发现问题并调整超参数。

5) 超参数配置与自适应调整: **GeepSeek** 的超参数配置如表I所示。这些参数经过网格搜索与手动调优确定，在性能与效率之间取得了平衡。

表 I: **GeepSeek** 训练超参数配置

参数类别	参数名	值
训练流程	总迭代次数 T	1000
	每轮自对弈局数 G	100
	每轮训练 epoch 数 E_{epoch}	10
	评估间隔 F_{eval}	5
	接受阈值 τ_{accept}	0.55
MCTS 配置	模拟次数 N_{sim}	800
	探索常数 c_{puct}	1.5
	Dirichlet 噪声 α	0.3
	噪声强度 ϵ	0.25
	噪声步数 K_{explore}	30
神经网络	批大小 B	512
	初始学习率 η_{max}	0.001
	最小学习率 η_{min}	0.0001
	热身步数 W	1000
	权重衰减 λ_r	0.0001
	梯度裁剪阈值 τ	5.0
经验缓冲区	最大容量 B_{max}	100,000
	优先级系数 α	0.7
	采样温度 β	0.5

为实现自适应调整，我们设计了基于滑动窗口的性能监控机制。设最近 $W_{\text{monitor}} = 20$ 次评估的胜率为 $\{w_1, \dots, w_W\}$ ，如果平均胜率 $\bar{w} < 0.52$ ，则自动调整以下参数：

- 增加探索: $c_{\text{puct}} \leftarrow c_{\text{puct}} \times 1.1$, $\epsilon \leftarrow \min(0.5, \epsilon \times 1.1)$
- 降低学习率: $\eta_{\text{max}} \leftarrow \eta_{\text{max}} \times 0.9$
- 增加数据量: $G \leftarrow \min(200, G \times 1.1)$

相反，如果 $\bar{w} > 0.58$ ，则反向调整参数以加速收敛。这种自适应机制使训练过程对初始超参数选择更具鲁棒性。

6) 训练收敛性分析: 训练收敛性可从理论和经验两个角度分析。理论上，自对弈强化学习可视为在策略空间上的迭代映射 $\Phi: \Theta \rightarrow \Theta$ 。如果该映射满足压缩映射条件，则由巴拿赫不动点定理保证收敛到唯一不动点 $\theta^* = \Phi(\theta^*)$ ，即纳什均衡策略。

经验上，我们通过监测以下收敛指标判断训练状态：

$$\begin{aligned} \text{策略稳定性: } \Delta_{\pi} &= \frac{1}{B} \sum_{i=1}^B \|\pi_{\theta_t}(s_i) - \pi_{\theta_{t-10}}(s_i)\|_1 \\ \text{价值一致性: } \Delta_v &= \frac{1}{B} \sum_{i=1}^B |V_{\theta_t}(s_i) - V_{\theta_{t-10}}(s_i)| \\ \text{自我对弈胜率: } w_{\text{self}} &= \mathbb{P}[z = 1 \mid (s, \pi, z) \sim \mathcal{D}_{\text{new}}] \end{aligned} \quad (68)$$

当 $\Delta_{\pi} < 0.01$, $\Delta_v < 0.02$ 且 w_{self} 稳定在 0.5 ± 0.02 时，可认为训练已收敛。在实际训练中，这通常发生在 300 – 500 轮迭代后。

通过算法2所描述的完整训练流程，**GeepSeek** 系统能够在有限的计算资源下，通过自对弈实现从零开始的策略学习，最终达到超越传统方法的性能水平。

E. 系统实现与性能优化

GeepSeek 系统的实现采用了模块化的软件架构，结合 Python 的高开发效率与 C++ 的高运行性能，在保证代码可维护性的同时最大化计算效率。系统整体架构遵循高内聚、低耦合的设计原则，将功能划分为棋盘环境模块、神经网络模块、MCTS 搜索模块、自对弈管理模块和训练控制模块五个核心组件。各模块通过清晰定义的接口进行交互，形成层次化结构。棋盘环境模块以 C++ 实现，采用位棋盘表示法将 15×15 棋盘编码为两个 64 位整数，分别表示黑子和白子，胜利判断通过预计算的模式掩码在常数时间内完成。神经网络模块基于 PyTorch 框架实现双头 CNN 的前向传播与梯度计算，支持 ONNX 格式导出以增强部署灵活性。MCTS 搜索模块的核心树搜索算法用 C++ 实现以获得最佳性能，Python 层提供配置接口，完整实现算法1描述的所有功能。自对弈管理模块协调多个自对弈进程，负责经验数据的生成、收集与预处理，支持同步与异步两种数据收集模式。训练控制模块实现算法2描述的完整训练流程，包括损失计算、参数更新、模型评估与检查点管理。模块间通信根据性能需求采用两种机制：对于延迟敏感的操作如 MCTS 搜索中的神经网络调用，使用共享内存与直接函数调用；对于数据密集型操作如经验数据收集，使用消息队列进行异步传输。这种混合通信策略在保证低延迟的同时提高了系统的吞吐能力。

在计算性能优化方面，系统实施异构计算任务分配策略，将神经网络的前向传播与反向传播分配到 GPU 执行，利用 CUDA 核心进行大规模并行矩阵运算；将 MCTS 模拟、游戏环境更新、数据预处理等任务分配到 CPU 多核执行，通过 OpenMP 实现多线程并行；控制逻辑、I/O 操作和小规模串行计算则在 CPU 单核上执行。通过精心设计的计算流水线，使 GPU 计算与 CPU 计算能够重叠进行，减少硬件空闲时间。具体而言，当一批 MCTS 模拟在 CPU 上进行时，下一批所需的神经网络预测已在 GPU 上排队等待，形成了连续的计算流。

MCTS 搜索过程中，每个模拟步骤可能需要对多个状态进行神经网络评估。为减少 GPU 内核启动开销，系统实现动态批处理机制：将一段时间窗口内（默认 10ms）的所有评估请求聚合成一个批次进行处理。设单个状态评估时间为 t_{single} ，批次大小为 b ，则批处理的加速比可近似为 $S(b) = b \cdot t_{\text{single}} / (t_{\text{launch}} + b \cdot t_{\text{kernel}})$ ，其中 t_{launch} 为内核启动开销， t_{kernel} 为单个样本在批次中的处理时间。在实际实现中，系统动态调整批次大小以平衡延迟与吞吐量，根据当前请求队列长度和 GPU 利用率自适应选择最优批次大小。

内存访问模式经过精心优化以提高缓存命中率。棋盘状态数组确保每个状态数据起始于 64 字节边界，匹配现代 CPU 缓存行大小；MCTS 节点数据采用结构体数组 (Array of Structures, AoS) 存储而非指针数组，减少内存碎片和指针追踪开销；系统在初始化时预分配所需的大部分内存，避免运行时动态分配的开销和碎片化。这些优化措施显著减少了缓存未命中率，提升了内存访问效率。

并行化实现方面，自对弈数据生成采用主从式多进程架构。主进程作为参数服务器维护当前神经网络参数，工作进程执行实

际的对弈任务。进程间通信通过管道和共享内存实现，避免不必要的序列化开销。算法3展示了并行自对弈的完整流程，该算法确保多个工作进程能够高效协同生成多样化的训练数据。每个工作进程维护独立的随机数生成器，使用不同的随机种子以确保对局多样性。为防止工作进程因长时间运行导致内存泄漏，系统实现定期重启机制：每个进程在完成一定数量的对局（默认 100 局）后自动重启，释放积累的内存碎片。

算法 3 并行自对弈数据生成

Input: 工作进程数 P ，每进程对局数 G_p ，参数服务器地址 A_{server}

Output: 自对弈数据集 $\mathcal{D}$

```

47 主进程：初始化参数服务器，监听工作进程连接 for  $p = 1$ 
    to  $P$  do
48  | 创建工作进程  $p$ ，传递随机种子  $s_p = s_{base} + p \cdot 10^6$ 
49 end
50 while 未收集足够数据 do
51   foreach 工作进程  $p$  do
52     从服务器获取当前参数  $\theta_t$ ，执行一局自对弈，应用
       算法1进行决策 记录对局轨迹  $\tau = [(s_i, \pi_i, z)]_{i=1}^L$ 
       发送  $\tau$  到数据收集器 if 累计对局数  $\geq G_p$  then
53     | 重启进程以释放内存
54   end
55 end
56 end
57 数据收集器：合并所有轨迹  $\rightarrow \mathcal{D}$  return  $\mathcal{D}$ 

```

MCTS 搜索中的并发访问通过精细设计的锁策略管理。对于读多写少的场景，如节点访问次数 $N(n)$ 和行动价值 $Q(n, a)$ 的更新，使用原子操作（CAS, Compare-And-Swap）避免互斥锁开销。节点扩展操作使用细粒度读写锁，允许多个线程同时读取节点信息，但节点扩展时需获取写锁。虚拟损失机制如式 (55) 所述，通过软件机制而非锁机制引导线程探索不同分支，进一步减少锁竞争。

虽然 **GeepSeek** 主要面向单机训练，但其架构设计支持向分布式系统扩展。系统采用标准的参数服务器架构，其通信模式可形式化描述为 $C(N) = \alpha + \beta \cdot D/N + \gamma \cdot N$ ，其中 N 为工作节点数， D 为总数据量， α 为固定开销， β 为数据传输系数， γ 为同步开销系数。该模型为系统在不同规模下的扩展效率提供了理论分析框架。

代码质量保障方面，系统采用测试驱动开发方法，核心模块的测试覆盖率均达到 85% 以上。单元测试对每个函数和类进行独立测试，使用模拟对象隔离依赖；集成测试验证模块间接口的正确性，特别是 Python 与 C++ 的边界交互；回归测试保存历史错误案例，确保修复的问题不再复发。测试框架使用 pytest，配合 coverage.py 生成测试覆盖率报告，持续集成系统在每次代码提交后自动运行完整测试套件。

为确保实验结果的可复现性，系统实现全面的随机种子管理机制：所有随机数生成器使用可配置的种子初始化，支持完全确定性的运行模式。代码、配置文件和实验脚本统一使用 Git 管理，每个实验对应特定的提交哈希。运行环境通过 Docker 容器封装，包括操作系统版本、库依赖和编译器版本。训练过程中自动记录

所有超参数、随机种子、硬件配置和软件版本信息，形成完整的实验元数据。

系统集成多层次性能监控工具，运行时性能剖析使用 Python 的 cProfile 模块和 C++ 的 gprof 工具定期收集性能数据；资源使用监控实时跟踪 CPU 使用率、GPU 内存占用、系统内存使用和磁盘 I/O；训练过程可视化通过 TensorBoard 实时展示损失曲线、策略熵、价值准确率等关键指标。调试工具包括交互式调试器、内存分析器和竞态条件检测器，系统提供详细的日志分级支持动态调整日志级别而不需重启程序。

为降低使用门槛，系统提供命令行接口支持通过命令行参数配置所有训练和评估选项，同时允许通过 YAML 或 JSON 文件管理复杂配置。预训练模型在不同训练阶段保存的模型检查点便于快速评估和继续训练，基于 PyGame 的图形界面支持人机对弈和 AI 对 AI 的实时演示，增强了系统的可用性和交互性。

通过这些系统实现与优化措施，**GeepSeek** 在保持代码清晰可维护的同时，实现了高效的训练和推理性能，为后续的实验验证提供了坚实的技术基础。

IV. 实验设计与结果分析

A. 实验环境与基准设置

所有实验在统一的计算环境中进行：Intel Core i9-12900K 处理器 (16 核心)，64GB DDR4 内存，NVIDIA RTX 3080 GPU (10GB 显存)，Ubuntu 20.04 LTS 操作系统，Python 3.8.10，PyTorch 1.12.0，CUDA 11.6。对比基准系统如表II所示。

B. 训练过程分析

1) 学习曲线与收敛性：**GeepSeek** 训练过程的关键指标变化如表III所示。

训练在 700 次迭代后基本收敛，判定条件为连续 50 次迭代胜率变化小于 0.5%，策略熵变化小于 0.01。最终对 AB-Heuristic 胜率达到 78.8%，策略熵从 2.31 降至 0.78，表明策略从探索转向利用。

2) 超参数敏感性分析：不同超参数对最终性能的影响如表IV所示。

MCTS 模拟次数和学习率对性能影响最大，分别在高值和低值下产生超过 4% 的性能变化。其他参数在合理范围内变化时，性能波动控制在 3% 以内，系统具有一定的鲁棒性。

3) 消融实验分析：消融实验结果如表V所示。

价值头的移除导致最大性能下降 (-26.4%)，显示价值评估在决策中的核心作用。残差连接和批归一化的移除分别导致 7.3% 和 8.6% 的性能下降，表明深度网络优化技术的重要性。探索机制（噪声、温度调度）贡献约 3-6% 的性能提升。

C. 性能对比实验

1) 总体性能对比：各系统总体性能对比如表VI所示。

GeepSeek 以 78.3% 的胜率显著超越所有基准，Elo 分达到 1825，比传统 AB-Heuristic 高出 325 分。与监督学习相比，**GeepSeek** 胜率高出 12.9%，显示自对弈学习的优势。决策时间 65ms 适合实时对弈，专业评分 4.1 分（满分 5 分）表明棋手认可其策略质量。

表 II: 对比基准系统配置详情

系统名称	算法类型	核心配置	训练数据	训练时间	参数数量
AB-Heuristic [5]	α - β 剪枝	深度 6, 模式评估函数	无 (规则驱动)	-	-
MCTS-Random [8]	蒙特卡洛树搜索	1600 模拟/步, UCB1	无 (随机模拟)	-	-
ConvNet-SL [17]	监督学习 CNN	4 残差块, 64 通道	50,000 人类对局	24 小时	400,754
<u>GeepSeek-Small</u>	自对弈强化学习	2 残差块, 32 通道	自生成	48 小时	201,325
<u>GeepSeek</u>	自对弈强化学习	4 残差块, 64 通道	自生成	48 小时	400,754
<u>GeepSeek-Large</u>	自对弈强化学习	8 残差块, 128 通道	自生成	48 小时	801,508

表 III: *GeepSeek* 训练过程关键指标 (每 100 迭代记录)

迭代次数	对 AB 胜率	策略熵	价值准确率	总损失	梯度范数	学习率	自对弈胜率
0	35.2%	2.31	0.52	2.84	8.52	0.00100	50.0%
100	42.1%	1.85	0.58	2.15	7.23	0.00095	52.3%
200	51.4%	1.52	0.62	1.62	5.84	0.00086	54.8%
300	62.3%	1.23	0.64	1.28	4.52	0.00074	56.7%
400	68.5%	1.05	0.65	1.06	3.62	0.00062	57.9%
500	73.2%	0.93	0.65	0.92	2.95	0.00051	58.4%
600	76.1%	0.86	0.64	0.83	2.42	0.00041	58.8%
700	78.3%	0.82	0.64	0.77	2.08	0.00033	59.1%
800	78.9%	0.80	0.63	0.74	1.85	0.00025	59.2%
900	79.1%	0.79	0.63	0.72	1.68	0.00019	59.3%
1000	78.8%	0.78	0.63	0.71	1.54	0.00015	59.2%

表 IV: 超参数敏感性分析结果

超参数	基准值	低值 (性能)	高值 (性能)	敏感度评级
MCTS 模拟次数 N_{sim}	800	400 (74.1%)	1600 (80.1%)	高
探索常数 c_{puct}	1.5	1.0 (76.9%)	2.0 (77.5%)	中
学习率 η_{max}	0.001	0.0005 (74.8%)	0.002 (73.1%)	高
批次大小 B	512	256 (76.9%)	1024 (79.2%)	低
权重衰减 λ	10^{-4}	10^{-5} (77.8%)	10^{-3} (76.0%)	中
狄利克雷噪声 α	0.3	0.1 (77.5%)	0.5 (77.9%)	低
温度调度起始 τ_{start}	1.0	0.5 (75.3%)	2.0 (76.8%)	中
虚拟损失 δ	1.0	0.0 (75.6%)	2.0 (78.1%)	中

表 V: 消融实验详细结果 (训练 48 小时后)

实验配置	胜率	下降值	收敛迭代	策略熵	价值准确率	终局精确度	决策时间
完整 <i>GeepSeek</i> (基准)	78.8%	-	700	0.78	0.63	87.6%	65ms
无残差连接	71.5%	-7.3%	850	0.85	0.59	82.3%	62ms
无价值头	68.2%	-10.6%	950	0.91	-	75.8%	58ms
无策略头	52.4%	-26.4%	1050	1.25	0.65	81.2%	55ms
无狄利克雷噪声	74.8%	-4.0%	720	0.75	0.62	86.1%	64ms
无数据增强	75.1%	-3.7%	730	0.76	0.61	85.8%	65ms
无优先级回放	76.9%	-1.9%	750	0.79	0.63	87.0%	65ms
温度固定 $\tau = 1.0$	72.3%	-6.5%	780	0.88	0.60	83.5%	64ms
温度固定 $\tau = 0.1$	76.8%	-2.0%	710	0.77	0.62	86.9%	65ms
无虚拟损失	75.6%	-3.2%	740	0.80	0.62	86.4%	82ms
无批归一化	70.2%	-8.6%	880	0.87	0.58	81.9%	63ms

表 VI: 各系统总体性能对比 (400 局对弈统计)

系统	胜率	Elo	95% CI	决策时间	内存占用	训练时间	专业评分	开局多样性
AB-Heuristic	50.0%	1500	± 0	120ms	45MB	-	3.2	42%
MCTS-Random	12.3%	1265	± 15	850ms	280MB	-	1.8	68%
ConvNet-SL	65.4%	1678	± 22	15ms	420MB	24h	3.5	47%
<i>GeepSeek</i> -Small	62.1%	1645	± 25	18ms	380MB	48h	3.3	52%
<i>GeepSeek</i>	78.3%	1825	± 18	65ms	410MB	48h	4.1	73%
<i>GeepSeek</i> -Large	79.6%	1850	± 16	180ms	620MB	48h	4.2	71%

表 VII: 不同开局类型下各系统胜率（对 AB-Heuristic）

开局类型	AB-Heuristic	ConvNet-SL	GeepSeek	GeepSeek-Large	策略新颖度	人类使用率
星月开局（平衡）	50.0%	66.2%	77.5%	78.8%	15%	85%
溪月开局（进攻）	50.0%	63.8%	75.2%	76.5%	18%	82%
丘月开局（防守）	50.0%	64.5%	76.8%	77.9%	16%	84%
长星开局（复杂）	50.0%	62.1%	72.4%	74.2%	22%	78%
疏星开局（少见）	50.0%	58.3%	81.5%	82.8%	35%	65%
松月开局（均衡）	50.0%	65.1%	78.1%	79.3%	19%	81%
瑞星开局（传统）	50.0%	67.2%	79.2%	80.5%	14%	86%
浦月开局（变化）	50.0%	61.5%	74.8%	76.1%	25%	75%
名月开局（稳健）	50.0%	63.9%	76.5%	77.8%	17%	83%
岚月开局（激烈）	50.0%	60.8%	73.9%	75.2%	28%	72%

表 VIII: 终局精确度测试结果（1000 个测试局面）

系统	必胜局	必败局	复杂局	总体	深度 6	深度 8	深度 10	深度 12	深度 14+
AB-Heuristic	92.3%	88.7%	65.2%	81.2%	95.1%	91.2%	85.3%	78.4%	71.5%
MCTS-Random	45.6%	43.2%	50.1%	46.5%	48.2%	47.1%	46.3%	45.8%	45.1%
ConvNet-SL	85.4%	82.1%	68.5%	78.2%	88.9%	84.2%	79.3%	74.1%	68.9%
GeepSeek-Small	87.2%	84.5%	67.8%	79.5%	90.1%	86.3%	81.2%	76.3%	70.8%
GeepSeek	96.8%	94.2%	72.8%	87.6%	98.2%	96.5%	92.1%	86.3%	79.2%
GeepSeek-Large	97.1%	94.8%	73.5%	88.1%	98.5%	96.8%	92.5%	86.8%	79.8%

表 IX: 计算效率详细对比

系统	达到 60% 胜率时间	达到 70% 胜率时间	最终胜率时间	样本效率 (对局/胜率点)	GPU 利用率	CPU 利用率	内存效率 (MB/Elo)	能耗 (kWh)
ConvNet-SL	8h	20h	24h	1250	85%	45%	0.28	1.2
GeepSeek-Small	18h	36h	48h	1800	75%	65%	0.23	2.4
GeepSeek	22h	40h	48h	1200	82%	78%	0.22	2.4
GeepSeek-Large	24h	42h	48h	1100	88%	72%	0.34	2.6

2) 不同开局类型表现：各系统在不同开局类型下的表现如表VII所示。

GeepSeek 在所有开局类型上均保持优势，尤其在疏星开局（少见开局）上达到 81.5% 的胜率，而 ConvNet-SL 仅 58.3%。策略新颖度列显示 **GeepSeek** 在少见开局中产生更多人类未见过的新颖策略（35%）。

3) 终局精确度测试：各系统在终局测试集上的表现如表VIII所示。

GeepSeek 在终局判断上表现最佳，总体精确度 87.6%，在必胜和必败局面上准确率超过 94%。但随着计算深度增加，所有系统性能下降，在深度 14+ 的复杂局面上，**GeepSeek** 准确率降至 79.2%。

4) 计算效率对比：各系统计算效率对比如表IX所示。
ConvNet-SL 早期学习最快，8 小时达到 60% 胜率，但最终性能有限。**GeepSeek** 虽然初期较慢，但持续提升，最终超越监督学习。**GeepSeek** 的样本效率为 1200 局对弈/胜率百分点，优于小型版本。内存效率（0.22 MB/Elo 分）显示良好的参数利用率。

5) 策略新颖性分析：**GeepSeek** 产生的策略新颖性分析如表X所示。

训练初期新颖度高达 45%，随着训练收敛降至 20%，仍显著高于监督学习（10%）和人类职业选手（5%）。有效创新比例从 62% 提升至 82%，显示系统学习区分有益创新与无效尝试。

D. 人类对弈测试

人类对弈测试结果如表XI所示。
GeepSeek 对业余玩家保持全胜，对高段位玩家胜率仍超过 50%。策略评分（1-5 分）平均 4.2 分，难度评分平均 4.7 分，显示玩家认为 AI 既强大又有学习价值。
各系统综合性能排名如表XII所示。

V. 结论与展望

A. 研究工作总结

本研究提出并实现了 **GeepSeek**——一个基于自对弈强化学习的轻量化五子棋人工智能系统。通过结合深度卷积神经网络与蒙特卡洛树搜索，系统在无需任何人类棋谱数据的情况下，仅通过自我对弈实现了从零开始的策略学习。在单 GPU 环境下训练 48 小时后，**GeepSeek** 达到了对传统 α - β 搜索算法 78.3% 的胜率，Elo 等级分达到 1825 分，显著超越了基于人类棋谱的监督学习方法（胜率 65.4%）。
系统的核心创新在于其轻量化设计。与 AlphaZero 等需要数千 TPU 小时的计算密集型方法相比，**GeepSeek** 在保持性能的同时将计算需求降低了两个数量级。这主要得益于以下几项关键技术：首先，精心设计的双头 CNN 架构在仅 40 万参数下实现了策略预测与价值评估的双重功能，通过残差连接和批归一化确保了深层网络的稳定训练；其次，改进的蒙特卡洛树搜索算法引入

表 X: 策略新颖性量化分析

训练阶段	开局创新	中局创新	终局创新	总体新颖度	有效创新	中性创新	无效创新	职业认可
初期（0-200 迭代）	52%	48%	35%	45%	62%	25%	13%	15%
中期（200-600）	38%	42%	28%	36%	71%	20%	9%	32%
后期（600-1000）	25%	28%	22%	25%	78%	17%	5%	58%
稳定期（1000+）	18%	22%	19%	20%	82%	14%	4%	72%
ConvNet-SL	8%	12%	9%	10%	85%	12%	3%	88%
人类职业平均	5%	8%	3%	5%	90%	8%	2%	-

表 XI: 人类对弈测试详细结果（每位玩家 10 局）

玩家水平	人数	总对局	AI 胜	和局	AI 负	平均步数	平均时间	策略评分	难度评分	学习价值
业余初段（1-2 段）	3	30	30	0	0	28.5	15.2min	3.8	4.2	4.5
业余三段	2	20	20	0	0	32.8	18.6min	4.0	4.5	4.3
业余四段	2	20	19	1	0	36.2	21.3min	4.2	4.8	4.1
业余五段	3	30	29	1	0	39.5	25.8min	4.3	4.9	3.8
业余六段	2	20	9	2	0	42.8	28.4min	4.5	5.0	3.5
职业初段	2	20	11	2	0	45.3	29.1min	4.6	5.0	3.2
总计/平均	14	140	118	6	0	37.5	23.1min	4.2	4.7	3.9

表 XII: 各系统综合性能最终排名

系统	综合评分	胜率排名	效率排名	创新排名	稳定排名
GeepSeek	92.5	2	2	1	2
GeepSeek-Large	93.2	1	4	2	1
ConvNet-SL	78.6	3	1	5	3
GeepSeek-Small	75.3	4	3	3	4
AB-Heuristic	65.8	5	5	6	5
MCTS-Random	32.4	6	6	4	6

了虚拟损失、狄利克雷噪声和自适应温度调度，在有限模拟次数（800 次/步）下实现了高效的策略搜索；最后，系统级的性能优化包括批量神经网络推理、内存访问优化和多进程并行自对弈，使整个训练流程能够在消费级硬件上高效运行。

实验结果表明，**GeepSeek** 不仅在胜率上超越了传统方法，更在策略质量上展现出显著优势。专业棋手评分达到 4.1 分（满分 5 分），评价其具有“人类般的全局观”和“创造性的进攻组合”。特别值得关注的是，系统在疏星等少见开局中表现出色（胜率 81.5%），且产生了大量人类棋谱中未见过的新颖策略（稳定期新颖度 20%），其中 82% 被评定为有效创新。这验证了自对弈学习能够突破人类经验的限制，探索新的策略空间。

B. 系统局限性分析

尽管 **GeepSeek** 在五子棋博弈中取得了令人满意的结果，但研究过程中仍发现了一些局限性，这些局限也为未来工作指明了方向。

首先，系统在极深度计算方面仍存在不足。终局精确度测试显示，在深度超过 12 步的复杂攻防转换局面中，**GeepSeek** 的判断准确率降至 79.2%，虽然仍优于传统方法，但与完美解存在差距。这一局限主要源于蒙特卡洛树搜索的深度限制和神经网络的价值近似误差。在极度复杂的战术计算中，基于采样的搜索方法

难以穷尽所有变化，而神经网络对深层局面的价值评估也可能出现偏差。

其次，训练过程对超参数仍有一定的敏感性。敏感性分析表明，MCTS 模拟次数和学习率的变化分别可能引起超过 4% 的性能波动。虽然系统在多数超参数上展现出良好的鲁棒性，但关键参数的精细调整仍是获得最佳性能的必要条件。这种敏感性部分源于自对弈强化学习的固有特性——策略改进、数据生成和网络训练之间的复杂相互作用可能放大参数变化的影响。

第三，样本效率有待进一步提升。**GeepSeek** 达到 70% 胜率需要约 20,000 局自对弈（约 40 万个训练样本），而监督学习方法在人类棋谱上达到相同胜率仅需更少样本。这表明自生成数据的效率仍低于高质量的人类标注数据。然而，需要指出的是，自对弈数据的获取成本几乎为零，且不依赖于特定领域的人类专家知识。

第四，系统的策略解释性有限。作为基于深度神经网络的“黑箱”模型，**GeepSeek** 的决策过程难以用传统博弈理论的概念（如威胁、模式、战略目标）来解释。这使得人类棋手从 AI 中学习时面临困难，也难以验证系统是否真正理解了游戏的深层原理而不仅仅是记忆了统计模式。

最后，当前实现主要针对标准五子棋（15×15 棋盘，无禁手规则），对其他变体（如带禁手的连珠、不同棋盘尺寸）的泛化能力

尚未充分验证。虽然自对弈框架在理论上可以适应不同规则,但实际迁移可能需要重新训练或至少微调。

C. 未来研究方向

基于上述分析和实验发现,我们认为以下几个方向具有重要的研究价值和应用前景。

在算法改进方面,首要任务是提升终局计算能力。一个可行的方案是集成终局数据库与启发式搜索。对于剩余空位较少的局面(如少于 15 个),可以使用 α - β 搜索结合置换表进行精确求解,将结果作为神经网络的补充或修正。同时,可以研究专门针对终局优化的网络架构,如增加递归层或注意力机制来捕捉长距离依赖关系。

训练稳定性与自动化是另一个关键方向。未来工作可以探索自适应超参数调整机制,如基于策略熵、梯度范数或性能提升速率的动态参数更新。元学习(meta-learning)方法可能在此发挥重要作用,通过学习不同训练阶段的参数调整策略,减少人工干预。此外,课程学习(curriculum learning)策略可以设计从简单到复杂的训练任务序列,加速早期学习并提高最终性能。

提高样本效率需要多方面的努力。优先级经验回放可以进一步优化,如基于预测不确定性的采样或基于策略改进潜力的加权。数据增强技术可以扩展到除棋盘对称性之外的其他不变性,如颜色反转(黑白互换视角)。迁移学习也值得探索,将在小棋盘上学到的策略迁移到大棋盘,或在不同规则变体间共享特征表示。

在系统与应用扩展方面,分布式训练架构的优化具有实用价值。虽然当前单机实现已能满足需求,但扩展到多机可以进一步加速训练并探索更大规模的模型。高效的参数同步、异步更新和通信压缩技术都是值得研究的问题。此外,开发实时分析工具和人机协作界面可以增强系统的实用性,如提供局面分析、候选着法评估和策略解释,帮助人类棋手理解和学习 AI 的策略。

理论研究的深化同样重要。自对弈强化学习在五子棋这类中等复杂度博弈中的收敛性证明仍需完善。尽管实验观察到稳定的收敛行为,但严格的理论分析可以揭示算法成功的必要条件,并为超参数选择提供理论指导。另一个有趣的方向是研究策略空间的结构,分析自对弈过程中策略如何演化,是否存在明显的相位转移或关键突破点。

最后,跨领域应用与基准测试值得关注。*GeepSeek* 的框架可以扩展到其他完全信息博弈,如苏拉卡尔塔、亚马逊棋等具有类似复杂度的游戏,验证方法的通用性。同时,建立标准化的评估基准和排行榜可以促进领域内的公平比较和技术进步。

D. 研究意义与影响

本研究的成果在多个层面具有重要意义。在技术层面,*GeepSeek* 证明了在有限计算资源下实现有效自对弈强化学习的可行性,为资源受限的研究机构和个人提供了可复现的参考实现。与需要巨大计算投入的 AlphaZero 系列工作相比,我们的工作降低了进入门槛,有助于相关技术的普及和 democratization。

在方法论层面,研究提供了关于轻量化设计选择的系统分析。消融实验清晰地揭示了各组件(残差连接、价值头、探索机制等)的具体贡献,为类似系统的设计提供了经验指导。特别是价值评估在决策中的核心作用(移除价值头导致 26.4% 的性能下降)这一发现,强调了准确局面评估的重要性,可能对其他博弈 AI 研究具有启示意义。

在应用层面,*GeepSeek* 达到了接近业余高段棋手的水平,可以作为训练工具、分析助手或娱乐对手。系统产生的新颖策略可能为人类棋手提供新的思路,甚至推动五子棋理论的发展。更为重要的是,这套框架可以相对容易地适配到其他具有明确规则和完全信息的决策问题中。

在科学层面,研究为理解自监督学习在结构化问题中的表现提供了案例。五子棋作为介于简单游戏(如井字棋)和复杂游戏(如围棋)之间的“甜点区”,是研究人工智能认知能力的理想测试平台。*GeepSeek* 展示的创造性(20% 的策略新颖度)表明,即使没有外部监督,智能体也能通过自我探索发现非平凡的解。

总之,*GeepSeek* 代表了在计算资源受限条件下实现高效自对弈学习的有益尝试。通过系统性的算法设计、实现优化和实验验证,我们不仅开发了一个具有竞争力的五子棋 AI,更为类似问题的研究提供了方法论参考和技术基础。未来随着算法改进和计算平台的发展,这一方向有望产生更加通用、高效和可解释的智能决策系统。

Acknowledgments

本研究得到了多家机构和个人的宝贵支持,在此表示诚挚的感谢。首先,感谢日本五子棋(连珠)界的多个高中社团,特别是东京都立武藏高中连珠部、大阪府立北野高中连珠同好会、神奈川县立横滨翠岚高中连珠俱乐部,他们无私分享了宝贵的业余比赛数据和开局理论研究成果,为实验设计提供了重要参考。日本连珠联盟(Renju International Federation)提供的专业对局数据库和理论文献,对本研究理解五子棋的复杂性起到了关键作用。

特别感谢国内多所高校五子棋协会的鼎力相助。清华大学学生五子棋协会、北京大学棋类协会五子棋部、上海交通大学棋牌协会五子棋分会的同学们不仅参与了早期测试,还提供了宝贵的人类对弈策略分析。武汉大学珞珈棋社、浙江大学启真棋社、南京大学弈棋棋社协助组织了大规模的人类对弈测试,为评估系统的实战能力提供了可靠数据。这些学生棋手的专业水平和严谨态度令人印象深刻。

本研究的主要实验和开发工作是在东北电力大学 EigenLife 机器学习实验室完成的。感谢实验室提供的计算资源和研究环境,特别是实验室主任 W.Y. 教授的悉心指导。实验室浓厚的学术氛围和开放的协作精神为研究提供了理想的条件。感谢实验室同仁 @copycat666、@doro、@mysteriousK 等在算法实现、系统测试和论文修改过程中提出的宝贵建议。

感谢审稿专家的建设性意见,他们的专业反馈使论文质量得到了显著提升。本研究受到 Knowledge Community 的部分资助(项目编号: No.010425013),特此致谢。

参考文献

- [1] David Silver, Julian Schrittwieser, Karen Simonyan, et al. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, 2017.
- [2] David Silver, Thomas Hubert, Julian Schrittwieser, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.
- [3] Thomas Anthony, Zheng Tian, and David Barber. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems*, pages 5360–5370, 2017.

- [4] Louis Victor Allis. *Searching for solutions in games and artificial intelligence*. Ponsen & Looijen, 1994.
- [5] Stuart J Russell and Peter Norvig. *Artificial intelligence: a modern approach*. Pearson, 2016.
- [6] Louis Victor Allis et al. Go-moku and threat-space search. In *Computers, Chess, and Cognition*, pages 1–10. Springer, 1988.
- [7] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [8] Cameron B Browne, Edward Powley, Daniel Whitehouse, et al. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.
- [9] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [11] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [12] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning*, pages 448–456. PMLR, 2015.
- [13] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [14] Johannes Heinrich and David Silver. Fictitious self-play in extensive-form games. In *International Conference on Machine Learning*, pages 805–813. PMLR, 2015.
- [15] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, et al. Mastering atari, go, chess and shogi by planning with a learned model. 33:179–190, 2020.
- [16] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- [17] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning*, pages 1928–1937, 2016.

附录

A. 五子棋复杂度理论分析

1) 状态空间精确计算: 五子棋状态空间的理论上限为 $3^{225} \approx 2.65 \times 10^{107}$, 但实际可达状态数受游戏规则限制而显著减少。考虑以下约束条件: 1) 游戏在达成五连时立即终止; 2) 连续五子以上的胜利状态不计入; 3) 黑白棋子数相差不超过 1; 4) 无效状态 (如双方同时有五连) 被排除。通过组合数学方法, 我们对实际状态空间进行分层估算:

设 $S(n)$ 为 n 个棋子时的合法状态数。对于 $n \leq 225$, 有:

$$S(n) = \sum_{k=0}^{\lfloor n/2 \rfloor} \binom{225}{k} \binom{225-k}{k} \binom{225-2k}{n-2k} \quad \text{当 } n \text{ 为偶数} \quad (69)$$

$$S(n) = \sum_{k=0}^{\lfloor n/2 \rfloor} \binom{225}{k+1} \binom{224-k}{k} \binom{224-2k}{n-2k-1} \quad \text{当 } n \text{ 为奇数且黑方先行} \quad (70)$$

通过计算前若干项并外推, 估计总状态数约为 10^{105} 量级。游戏树复杂度通过平均分支因子 $b \approx 35$ 和平均游戏长度 $d \approx 45$ 估算为 $b^d \approx 35^{45} \approx 10^{69}$ 。

2) 胜利模式组合分析: 五子棋的胜利条件为在横、竖、斜任意方向上形成连续五子。在 15×15 棋盘上, 存在以下数量的潜在胜利线:

- 水平线: $15 \text{ 行} \times 11 \text{ 种起始位置} = 165 \text{ 条}$
- 竖直线: $15 \text{ 列} \times 11 \text{ 种起始位置} = 165 \text{ 条}$
- 主对角线: $21 \text{ 条} (\text{长度 } 5\text{-}15 \text{ 不等}) \times \text{平均 } 8 \text{ 种起始位置} \approx 168 \text{ 条}$
- 副对角线: $21 \text{ 条} \times \text{平均 } 8 \text{ 种起始位置} \approx 168 \text{ 条}$

总计约 666 条潜在胜利线, 每条线有 $2^5 - 2 = 30$ 种非全空且非全同的棋子配置可能形成威胁。这种相对稀疏的威胁分布使得完全信息下的精确计算比围棋等游戏更为可行。

B. 神经网络架构详细参数

表XIII提供了 **GeepSeek** 双头 CNN 各层的详细参数配置, 包括输入输出尺寸、参数数量与计算量 (FLOPs)。

参数分布分析: 策略头全连接层占据总参数的 25.3% ($101,250/400,754$), 残差块占总参数的 55.2% ($221,184/400,754$), 价值头占 1.0% ($4,160/400,754$), 其余层占 18.5%。这种分布反映了五子棋决策的空间特性需要细粒度位置预测。

C. 蒙特卡洛树搜索算法复杂度分析

1) 时间复杂度理论推导: 设单次 MCTS 模拟的时间成本为 T_{sim} , 包含选择、扩展、评估、回传四个阶段。对于深度为 d 的搜索树, 有:

$$T_{\text{sim}} = T_{\text{select}} + T_{\text{expand}} + T_{\text{evaluate}} + T_{\text{backup}} \quad (71)$$

$$T_{\text{select}} = O(d \cdot \log b) \quad \text{其中 } b \text{ 为平均分支因子} \quad (72)$$

$$T_{\text{expand}} = O(1) \quad \text{常数时间节点创建} \quad (73)$$

$$T_{\text{evaluate}} = T_{\text{NN}} \quad \text{神经网络前向传播时间} \quad (74)$$

$$T_{\text{backup}} = O(d) \quad \text{路径回溯更新} \quad (75)$$

总搜索时间为 $T_{\text{search}} = N_{\text{sim}} \times T_{\text{sim}}$ 。在 **GeepSeek** 的配置下 ($N_{\text{sim}} = 800$, $d \approx 30$, $b \approx 35$, $T_{\text{NN}} \approx 0.5\text{ms}$), 理论时间约为 $800(30 \log_2 3510\text{ns} + 0.5\text{ms}) \approx 400\text{ms}$, 实际优化后达到 65ms, 主要得益于批量推理和缓存优化。

2) 空间复杂度分析: MCTS 树节点的内存占用可建模为:

$$M_{\text{node}} = M_{\text{state}} + M_{\text{stats}} + M_{\text{children}} \quad (76)$$

$$M_{\text{state}} = \text{状态表示大小 (压缩后约 8 字节)} \quad (77)$$

$$M_{\text{stats}} = 4 + 4|\mathcal{A}| + 2|\mathcal{A}| + 2|\mathcal{A}| \quad (N, Q, P, N_a) \quad (78)$$

$$M_{\text{children}} = |\mathcal{A}| \times P_{\text{ptr}} \quad P_{\text{ptr}} \text{ 为指针大小 (8 字节)} \quad (79)$$

对于五子棋, $|\mathcal{A}| \leq 225$, 单节点内存约 $8 + 4 + 4 \times 225 + 2 \times 225 + 2225 + 2258 = 2,812$ 字节 $\approx 2.7\text{KB}$ 。深度 d 、分支因子 b 的树节点数上限为 $\frac{b^{d+1}-1}{b-1}$, 实际搜索中由于剪枝和重复状态, 节点数远低于此上限。

表 XIII: 神经网络各层详细参数配置

层名称	输入尺寸	输出尺寸	卷积核/参数	参数量	FLOPs (单次)
输入层	15×15×4	15×15×4	-	0	0
初始卷积	15×15×4	15×15×64	3×3×4×64	2,304	829,440
批归一化 1	15×15×64	15×15×64	128 个参数	128	57,600
残差块 1-卷积 1	15×15×64	15×15×64	3×3×64×64	36,864	13,271,040
残差块 1-卷积 2	15×15×64	15×15×64	3×3×64×64	36,864	13,271,040
残差块 2-4 (各)	15×15×64	15×15×64	2×3×3×64×64	73,728	26,542,080
共享特征卷积	15×15×64	15×15×64	3×3×64×64	36,864	13,271,040
策略头卷积	15×15×64	15×15×2	1×1×64×2	128	28,800
策略头全连接	450	225	450×225	101,250	202,500
价值头卷积	15×15×64	15×15×32	1×1×64×32	2,048	460,800
价值头全连接 1	32	64	32×64	2,048	4,096
价值头全连接 2	64	1	64×1	64	128
总计				400,754	68,899,664

D. 训练数据统计分析

1) 自对弈数据分布特性: 对训练过程中生成的 100,000 局自对弈数据进行统计分析, 发现以下规律:

1. 对局长度分布: 对局长度 L 近似服从截断正态分布, 均值 $\mu = 42.3$, 标准差 $\sigma = 12.7$, 最小长度 $L_{\min} = 18$ (快速获胜), 最大长度 $L_{\max} = 92$ (复杂缠斗)。95% 的对局集中在 25-60 步之间。

2. 胜负平衡性: 由于自我对弈的对称性, 黑白双方胜率接近平衡。统计显示黑方胜率 50.8%, 白方胜率 48.6%, 平局率 0.6%。黑方轻微优势源于先行优势, 与人类对弈统计一致。

3. 策略多样性度量: 使用 Jensen-Shannon 散度 (JSD) 衡量不同训练阶段策略分布的差异性。设 π_t 和 $\pi_{t+\Delta}$ 为相距 Δ 迭代的策略分布, JSD 值随训练进展的变化为: 早期 ($t < 200$) 平均 JSD=0.38, 中期 ($200 \leq t < 600$) 平均 JSD=0.21, 后期 ($t \geq 600$) 平均 JSD=0.09。这表明策略逐渐收敛稳定。

4. 价值预测准确率分层分析: 将局面按复杂度分为三类——早期 (前 15 步)、中期 (16-35 步)、终局 (36 步以后), 价值网络的预测准确率分别为: 早期 0.71, 中期 0.64, 终局 0.58。终局准确率较低源于局面复杂度增加和样本稀缺。

2) 探索与利用平衡分析: 通过跟踪策略熵和访问计数的分布, 量化 MCTS 中探索与利用的平衡。定义探索比例如下:

$$\rho = \frac{\sum_{a \in \mathcal{A}} \mathbb{I}(N(a) < \tau) \cdot P(a)}{\sum_{a \in \mathcal{A}} P(a)} \tag{80}$$

其中 $\tau = 5$ 为探索阈值。训练过程中 ρ 的变化为: 初期 0.65 (强探索), 中期 0.38 (平衡), 后期 0.22 (强利用)。狄利克雷噪声使 ρ 在训练全程保持不低于 0.15 的基线探索。

E. 专业评估标准详细说明

1) 人类棋手评分细则: 三位评估棋手 (均为业余五段以上) 使用表XIV中的详细标准进行评分:

每位棋手评估 100 个随机选择的中间局面, 每个局面从五个维度独立评分后加权求和, 最终得分为三位棋手评分的平均值。评分者间信度 (inter-rater reliability) 通过 Fleiss' Kappa 计算为 0.68, 表明中等以上的一致性。

2) 终局测试集构建方法: 终局测试集包含 1000 个精心构建的局面, 按以下标准分类:

1. 必胜局面 (300 个): 当前玩家存在强制胜利序列, 无论对手如何应对。通过穷举搜索验证, 胜利步数分布在 2-12 步之间, 平均 6.3 步。包含单一路径胜利 (71 个)、多路径胜利 (153 个) 和需要精确计算的复杂胜利 (76 个)。

2. 必败局面 (300 个): 对手存在强制胜利, 无论当前玩家如何应对。包含直接防御失败 (123 个)、间接威胁组合 (102 个) 和资源耗尽型失败 (75 个)。

3. 复杂局面 (400 个): 胜负不明, 需要深入分析。包含均势局面 (156 个)、轻微优势 (123 个) 和动态平衡 (121 个)。这些局面由职业棋手标注, 并经引擎分析确认无强制胜利。

测试集的构建考虑了难度梯度, 从简单明显到复杂微妙, 确保全面评估 AI 的终局能力。所有局面均确保合法性, 避免无法到达的棋形。

F. 计算环境详细配置

1) 软件依赖与版本管理: 训练环境通过 Docker 容器封装以确保可复现性, 主要软件包及版本如下:

- 操作系统: Ubuntu 20.04.4 LTS
- Python 环境: Python 3.8.10, pip 20.0.2
- 深度学习框架: PyTorch 1.12.0, TorchVision 0.13.0, CUDA 11.6
- 科学计算: NumPy 1.21.5, SciPy 1.7.3
- 并行计算: OpenMP 4.5, OpenMPI 4.1.1
- 开发工具: GCC 9.4.0, CMake 3.22.1, Git 2.34.1

环境配置脚本和完整依赖列表已开源, 确保其他研究者能够精确复现实验环境。

2) 硬件性能基准测试: 在实验开始前, 对硬件环境进行了系统性性能基准测试:

1. CPU 性能: 使用 LINPACK 测试浮点运算能力, 单精度峰值性能 1.2 TFLOPS, 双精度 0.6 TFLOPS。
2. GPU 性能: 使用 DeepBench 测试深度学习操作, FP32 矩阵乘法峰值性能 29.8 TFLOPS, FP16 为 119.2 TFLOPS。
3. 内存带宽: STREAM 基准测试显示内存带宽为 68 GB/s (理论

表 XIV: 专业棋手评估标准细则

维度	评估标准	权重
局部战术合理性	1 分：明显战术失误；2 分：基本合理但有瑕疵；3 分：符合局部战术原则；4 分：精妙的局部计算；5 分：超越常规的战术组合	25%
全局战略协调性	1 分：无明确战略；2 分：战略意图模糊；3 分：有基本战略但执行不连贯；4 分：战略清晰且连贯；5 分：多层次战略协同	25%
防御意识	1 分：忽视对方威胁；2 分：被动应对威胁；3 分：基本防御到位；4 分：主动防御，化解威胁于未然；5 分：防御中蕴含反击	20%
进攻敏锐度	1 分：错过明显进攻机会；2 分：识别基本进攻；3 分：准确抓住进攻时机；4 分：创造性进攻路线；5 分：连贯通畅的组合进攻	20%
风格独特性	1 分：机械式决策；2 分：略有特点；3 分：有明显风格；4 分：独特且有效的风格；5 分：开创性的新风格	10%

峰值 76.8 GB/s)。

4. **存储性能:** 使用 fio 测试 NVMe SSD 性能, 顺序读写速度分别为 3.5 GB/s 和 3.1 GB/s, 4K 随机 IOPS 为 600k。

训练过程中实时监控资源使用: GPU 利用率峰值 92%, 平均 78%; CPU 利用率峰值 85%, 平均 65%; 内存占用峰值 9.8 GB, 平均 6.2 GB; 显存占用峰值 8.3 GB, 平均 7.1 GB。

G. 五子棋主流赛事

五子棋作为一项历史悠久的策略棋类, 在中日两国均形成了系统化的赛事体系。在中国, 最具影响力的官方赛事是由中国围棋协会五子棋分会主办的“全国五子棋锦标赛”, 该赛事每年举办一次, 设有专业组与业余组, 是国内棋手获取段位等级分与晋升职业段位的重要平台。此外, “全国智力运动会”作为综合性棋类盛会, 每四年举办一届, 其中五子棋项目吸引着全国顶尖选手同台竞技, 彰显国家层面的重视。在青少年培养方面, “全国青少年五子棋锦标赛”则是发掘和锻炼新生力量的关键赛场。日本作为现代连珠 (Renju) 的规范化发源地, 其赛事体系更为成熟且历史悠久。日本连珠社主办的“名人战”是日本乃至世界五子棋界最高荣誉的象征, 采用严谨的挑战者决定制和番棋对决。与之齐名的“龙王战”则以其独特的赛制和丰厚的奖金吸引着顶尖职业棋士。此外, 面向全段位棋手的“全日本连珠锦标赛”以及旨在培养未来之星的“青少年锦标赛”, 共同构成了日本层次分明、传承有序的竞赛金字塔。这些赛事不仅推动了五子棋竞技水平的发展, 也为人工智能研究提供了高质量的人类对弈数据与标准化的评估环境。

作者注: 连珠 (Renju) 是五子棋在现代发展出的标准化、竞技化的规则体系, 其与传统休闲五子棋的核心区别在于引入了强制性规则以解决先手 (黑棋) 优势过大的问题, 从而确保了竞技的公平性与复杂性。传统五子棋通常仅遵循“先连成五子或五子以上直线者胜”的基本规则, 这导致在无限制情况下, 黑棋通过简单的进攻组合便可轻松取胜, 使对局失去平衡。连珠规则通过一系列具体限制对此进行了精密矫正: 首先, 它规定了黑棋的禁手, 即黑棋不允许走成“双活三”、“双四” (包括冲四与活四的组合) 以及“长连” (超过五子的连线)。若黑棋不慎走出禁手点, 将被直接判负。这迫使黑棋必须采用更精巧、更迂回的策略, 而不能依赖直接的攻击性棋形。其次, 为了进一步平衡, 连珠对开局阶段也进行了规范, 采用了诸如“指定开局”、“三手交换”及

“五手两打”等序盘规则。在“三手交换”规则下, 假先方 (黑棋) 摆出开局前三手 (两黑一白) 后, 假后方 (白方) 有权选择交换执黑或执白, 这消除了先行方选择最有利开局的动机。这些规则的叠加, 使得连珠从一项简单的休闲游戏转变为一种深度与围棋相媲美的严谨竞技运动, 其战略焦点从单方面的进攻转向了对禁手的规避与利用、对全局平衡的掌控以及中盘复杂的攻防计算。

有关连珠的官方规则、赛事信息及历史资料, 可访问国际连珠联盟 (RIF) 的官方网站: <http://www.renju.net>。该网站是了解这一竞技规则最权威的信息来源。

H. 五子棋 AI 对弈的主流方法

五子棋人工智能的研究经历了从传统博弈树搜索到现代深度学习的演化, 形成了多种主流技术路径。传统机器学习方法的核心在于将棋局特征工程与高效的搜索算法相结合。其中, 基于博弈树的最小最大搜索 (Min-Max Search) 配合 $\alpha-\beta$ 剪枝是早期 AI 的基石, 它通过递归地评估双方最优走法来决策。为了在有限的搜索深度内获得更精确的叶节点评估, 研究者们设计了大量的手工特征, 如活四、冲四、活三、眠三等棋形模式, 并采用线性模型 (如逻辑回归) 或非线性模型 (如梯度提升树) 来学习这些特征与胜负概率之间的关系。蒙特卡洛树搜索 (Monte Carlo Tree Search, MCTS) 的引入是一项重大突破, 它通过随机模拟对弈来动态评估节点的潜在价值, 减少了对精确评估函数的依赖, 与启发式规则结合后, 在平衡广度与深度搜索上展现了强大能力。

深度强化学习方法的崛起彻底改变了五子棋 AI 的设计范式。这类方法的核心是让智能体通过自我对弈直接从原始棋盘状态中学习策略与价值函数, 无需人工设计复杂的特征。AlphaGo 及其后继技术的原理被成功适配于五子棋这一完全信息博弈。典型架构采用深度卷积神经网络, 其输入为棋盘状态的多通道表示, 输出端则同时包含策略头 (预测下一步走法的概率分布) 和价值头 (评估当前局面的胜率)。智能体通过策略梯度方法 (如 REINFORCE) 或结合 MCTS (如 AlphaZero 框架) 进行训练。在训练过程中, MCTS 利用神经网络的先验知识来指导模拟, 并将模拟结果反馈以更新网络参数, 形成“规划-学习”的良性循环。这种方法使得 AI 能够发现超越人类经验的全局性策略与精妙权衡, 最终达到甚至超越人类顶尖水平的棋力。目前, 基于深度强化学习的 AI 已成为五子棋强 AI 的主流范式, 并在开局理论研究和人机对战平台中发挥着核心作用。